UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros Industriales



Implementación de esquemas de seguridad Post-Cuánticos en sistemas embebidos para IoT

T R A B A J O D E F I N D E M Á S T E R

Presentado para la obtención del título de Máster por:

César Augusto Zerpa Iraci

Madrid, 2025

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros Industriales

Máster Universitario en Electrónica Industrial

Implementación de esquemas de seguridad Post-Cuánticos en sistemas embebidos para IoT

T R A B A J O D E F I N D E M Á S T E R

Presentado para la obtención del título de Máster por:

César Augusto Zerpa Iraci

Bajo la supervisión de:
Dr. Jorge Portilla

Madrid, 2025

<Dedicatoria (opcional)>

Agradecimientos

<Página de agradecimientos (opcional)>

< Si el trabajo ha recibido financiación de alguna convocatoria competitiva, por favor, indíquelo aquí escribiendo: "Este trabajo ha sido parcialmente apoyada por...". De lo contrario, elimine este texto para dejar esta sección en blanco. >

Abstract

<Resumen en inglés: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Resumen

<Resumen en español: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Índice general

Agradecimientos . . . iii
Abstract . . . v
Resumen . . . vi
Índice de figuras . . . viii
Índice de tablas . . . ix

1 Introducción **1**

2 Estado del arte **3**
2.1 Objetivos . . . 3
2.2 Marco Teórico . . . 4
2.2.1 Criptografía clásica y post-cuántica . . . 4
2.2.2 HQC . . . 10
2.2.3 Sistemas embebidos e IoT . . . 11
2.2.4 Criptografía post-cuántica en sistemas embebidos e IoT . . . 15
2.3 Trabajos Relacionados . . . 16
2.3.1 Code-based Cryptography in IoT: A HW/SW Co-Design of HQC . . . 16
2.3.2 PQM4: Testing and Benchmarking NIST PQC on ARM Cortex-M4 . . . 17
2.3.3 Optimized Implementation of HQC on Cortex-M4 . . . 17
2.3.4 A Comparative Study of Post-Quantum Cryptosystems for Internet-of-Things Applications . . . 18

3 Diseño e implementación del sistema **21**
3.1 Plataforma experimental . . . 21
3.2 Arquitectura del sistema . . . 23
3.3 Protocolo de intercambio y transporte de HQC . . . 27
3.4 Sesiones . . . 31
3.5 Cifrado de datos . . . 31
3.6 Sistema de adquisición de métricas . . . 33
3.7 Corrección del subsistema de comunicaciones . . . 33
3.7.1 Modificacion del driver . . . 34
3.8 Diseño experimental . . . 35
3.8.1 Escenarios y topologías . . . 35
3.8.2 Condiciones experimentales . . . 35
3.8.3 Métricas evaluadas . . . 35

3.8.4 Procedimiento experimental 35

4 Resultados **37**
4.1 P2P 37
4.2 Estrella 39
4.3 Línea 41
4.4 Árbol 43

5 Conclusiones **47**

Anexos **53**

Índice de figuras

2.1 Triada de la seguridad 5
2.2 Cifrado Simétrico 6
2.3 Cifrado Asimétrico 7
2.4 Modelo OSI. 13

3.1 Componentes de hardware utilizados en el proyecto. 22
3.2 Arquitectura general del sistema. 23
3.3 Arquitectura del sistema. 24
3.4 Topologías implementadas. 25
3.5 Arquitectura de fragmentación. 27
3.6 Arquitectura de reensamblado de fragmentos. 28
3.7 Arquitectura de estados. 29
3.8 Arquitectura de transferencia de mensajes. 30

4.1 Análisis global del tráfico generado en escenarios P2P 38
4.2 Análisis global del tráfico generado en escenarios estrella 39
4.3 Fragmentos recuperados NACK 41
4.4 Análisis global del tráfico generado en escenarios linea 42
4.5 Fragmentos recuperados NACK 43
4.6 Análisis global del tráfico generado en escenarios linea 45
4.7 Fragmentos recuperados NACK 46

Índice de tablas

3.1 Campos y funciones de la cabecera del protocolo. 26
3.3 Tipos de mensajes, identificadores y funciones en el protocolo. 26
3.5 Sesiones 31

Capítulo 1

Introducción

En los últimos años, los dispositivos IoT han ganado una importancia significativa tanto en la industria como en la vida cotidiana, impulsados por el crecimiento del número de usuarios de Internet. Se estima que la cantidad de dispositivos IoT alcanzará los 18 mil millones en los próximos años, con proyecciones que indican que para 2030 podrían superar los 40 mil millones [1].

Paralelamente, la computación cuántica ha experimentado avances significativos. Estas nuevas arquitecturas computacionales, que aprovechan los principios de la mecánica cuántica, son capaces de resolver problemas matemáticos de alta complejidad en tiempos inalcanzables para los ordenadores clásicos. Como consecuencia, los sistemas de cifrado ampliamente utilizados en la actualidad, como RSA y ECC, podrían volverse inseguros ante la capacidad de los futuros ordenadores cuánticos de resolver en poco tiempo problemas que antes se consideraban intratables.

Ante esta amenaza, el **Instituto Nacional de Estándares y Tecnología (NIST, por sus siglas en inglés)** inició un proceso de estandarización de algoritmos criptográficos resistentes a ataques cuánticos, con el objetivo de garantizar la seguridad de la información en el futuro [2].

La seguridad en los dispositivos IoT es un desafío crítico, ya que muchos de estos dispositivos dependen de algoritmos criptográficos tradicionales que podrían volverse obsoletos. Esto representa un riesgo significativo, especialmente en entornos donde los dispositivos embebidos operan con recursos computacionales limitados y no siempre pueden ser actualizados fácilmente. En este contexto, la **criptografía post-cuántica (PQC, Post-Quantum Cryptography)** se presenta como una solución necesaria para garantizar la seguridad a largo plazo, proporcionando resistencia a ataques cuánticos sin requerir hardware extremadamente potente.

Dentro de la criptografía post-cuántica, los algoritmos pueden clasificarse en varias categorías según el problema matemático en el que basan su seguridad [3]: Criptografía basada en códigos, Criptografía basada en redes euclidianas, Criptografía basada en funciones hash, Criptografía basada en isogenias de curvas elípticas, Criptografía basada en polinomios multivariantes. Durante el proceso de estandarización de criptografica post-cuántica del NIST, se evaluaron

diferentes codigos candidatos, **HQC**, **BIKE** y **Classic McEliece**. Tras la cuarta ronda de evaluación, en marzo de 2025 se selecciono **HQC** para su estandarización.

Este trabajo se centrará en el algoritmo **HQC**, un esquema de encapsulación de claves basado en códigos diseñado para proporcionar seguridad frente a ataques tanto de ordenadores clásicos como cuánticos [4].

En comparación con los otros esquemas finalistas, **HQC** ofrece un equilibrio entre tamaño de clave, eficiencia en el cifrado/descifrado y requisitos computacionales, lo que lo hace una alternativa interesante para dispositivos IoT. Según [5], **HQC** presenta un rendimiento adecuado para dispositivos con bajas prestaciones y consumo reducido, características esenciales en muchas aplicaciones IoT.

Capítulo 2

Estado del arte

El desarrollo de este trabajo se enmarca en los esfuerzos impulsados por **NIST** para estandarizar algoritmos criptográficos resistentes a ataques cuánticos.

En las fases iniciales del proyecto, se realizó un análisis comparativo de las propuestas criptográficas consideradas en la cuarta ronda del proceso de estandarización de **NIST**. Como resultado de este análisis, se seleccionó el esquema **HQC** como el esquema estudiado. Posteriormente, la selección de **HQC** por parte de **NIST** en marzo de 2025, reforzó su interes de an alizar su comportamiento en plataformas IoT.

Este capitulo se presenta como una revisón del estado del arte en el campo de la criptografía post-cuántica. En primer lugar, se introduce los conceptos esenciales de criptografía clásica y post-cuántica, prestando atencion a los mecanismos de encapsulación de claves y al esquema HQC. A continuacion, se describen las principales caracteristicas y limitaciones de las redes IoT que condicionan la incorporación de esquemas criptográficos post-cuánticos. Finalmente, se revisan los trabajos erlacionados relevantes sobre implementacion y evaluacion de esquemas post-cuánticos en dispositivos de recursos limitados.

2.1 Objetivos

El presente trabajo estudia la implementación de esquemas de seguridad post-cuánticos en sistemas embebidos para IoT, con el fin de evaluar su rendimiento en esta clase de entornos. Es por ello que se plantean una serie de objetivos que permiten estructurar el desarrollo de este proyecto.

1. Implementar el esquema **HQC** en un entorno IoT
2. Evaluar el rendimiento del esquema HQC en terminos de ejecucion, consumo de memoria, y tamaño de clave/cifrado en el sistema elegido.
3. Identificar posibles limitaciones y oportunidades de mejora para aplicaciones reales en dispositivos de bajo consumo
4. Evaluar la sobrecarga que introduce el uso del esquema HQC en las comunicaciones, en

comparación con un sistema sin cifrado post-cuántico.

2.2 Marco Teórico

2.2.1 Criptografía clásica y post-cuántica

La criptografía comienza con la historia, con el origen de la escritura. Los egipcios eran capaces de comunicarse mediante mensajes escritos con jeroglíficos; este código era el secreto de los escribas. La comunicación con códigos secretos era comúnmente necesaria para la diplomacia, los gobiernos tenían que comunicarse con sus embajadas remotas en entornos sospechosos durante la guerra. Los cuarteles generales debían comunicarse en entornos hostiles. Sin embargo, durante la mayoría de la historia se utilizaba la criptografía de manera pedestre. La mayoría de los códigos secretos tenían una seguridad basada en la oscuridad, las personas que querían ocultar su mensaje debían elegir su propio código secreto.

El punto de inflexión en la historia de la criptografía ocurrió en la segunda guerra mundial, con el uso de la maquina **Enigma**, la cual fue empleada para cifrar y descifrar comunicaciones. Su ruptura por parte del matemático **Alan Turing**, marcó el inicio en la criptografía, donde el análisis matemático y la computación juegan un papel fundamental [6].

Con el paso del tiempo, la criptografía dejó de ser un conjunto de técnicas empíricas basadas en la ocultación, para convertirse en una disciplina científica con fundamentos matemáticos sólidos.

Con este contexto, la criptografía se puede definir como la ciencia que se encarga de la seguridad de la información y las comunicaciones, a través de mecanismos como la autenticación y el cifrado. **NIST Computer Security Handbook** define el término de **seguridad informática**, como la protección otorgada a un sistema de información automatizado para alcanzar objetivos de preservar la integridad, disponibilidad y confidencialidad de los recursos del sistema de información [7].

Estas propiedades conforman la triada de la seguridad, la cual establece principios esenciales para proteger los sistemas informáticos:

- **Confidencialidad**: Garantiza que la información privada no se ponga en disposición de personas no autorizadas. Asegura que las personas que controlan en qué información relacionada con ellos puede ser recopilada y almacenada y por quién y a quién puede ser revelada esa información.
- **Integridad**: Asegura que la información y los programas se cambien solo de una manera especificada y autorizada. Garantiza que los sistemas se comporten de la manera en la que fueron pensados de manera intacta, y libre de manipulaciones no autorizadas.
- **Disponibilidad**: Garantiza que el sistema trabaje de manera correcta y no se deniega el servicio a los usuarios autorizados.

El **criptoanálisis** es la ciencia —y en ocasiones, el arte—, de descifrar criptosistemas sin conocer previamente la clave. El criptoanálisis es de vital importancia para los criptosistemas modernos, ya que sin el análisis constante por parte de expertos que intenten vulnerarlos, no

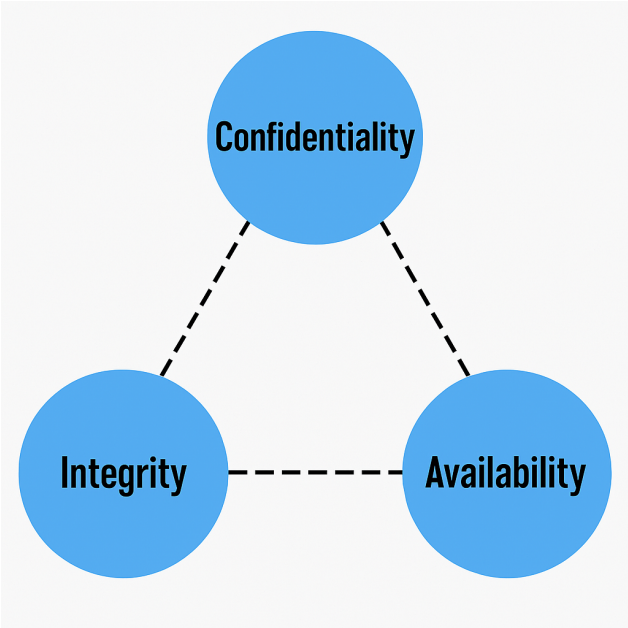


Figura 2.1: Triada de la seguridad

sería posible determinar si dichos métodos son verdaderamente seguros [8].

La **criptografía clásica** -también conocida como criptografía precuántica- se basa en algoritmos cuya seguridad radica en problemas matemáticos considerados intratables para los ordenadores convencionales. Estos algoritmos han resistido con éxito numerosos intentos de criptoanálisis a lo largo del tiempo. Entre los más representativos se encuentran:

- **RSA**, basado en la factorizacion de numeros enteros muy grandes.
- **Elliptic Curve Cryptography (ECC)**, que utiliza el logaritmo discreto en curvas elípticas.

La criptografía se divide, a su vez, en tres ramas principales:

- Algoritmos simétricos, utilizan una misma clave para cifrar y descifrar la información. Fueron el enfoque predominante desde los orígenes de la criptografía. Entre ellos, destaca AES por su eficiencia y robustez.
- Algoritmos asimétricos, introducidos en 1976 por **Whitfield Diffie**, **Martin Hellman** y **Ralph Merkle**, emplean un par de claves: una pública (que se puede compartir) y una privada (que debe mantenerse secreta). Ejemplos clave son RSA y ECC.
- Protocolos criptográficos, combinan estos algoritmos para lograr funciones como el intercambio seguro de claves, la autenticación o la firma digital, siendo componentes esenciales en aplicaciones como VPNs, correo cifrado o blockchain.

El cifrado simétrico habilita una comunicación segura sobre un canal de transmisión inseguro, asumiendo que la clave secreta se comparte sobre un canal seguro.

En la Figura 2.2 se observa cómo la clave secreta se transmite del receptor al transmisor de

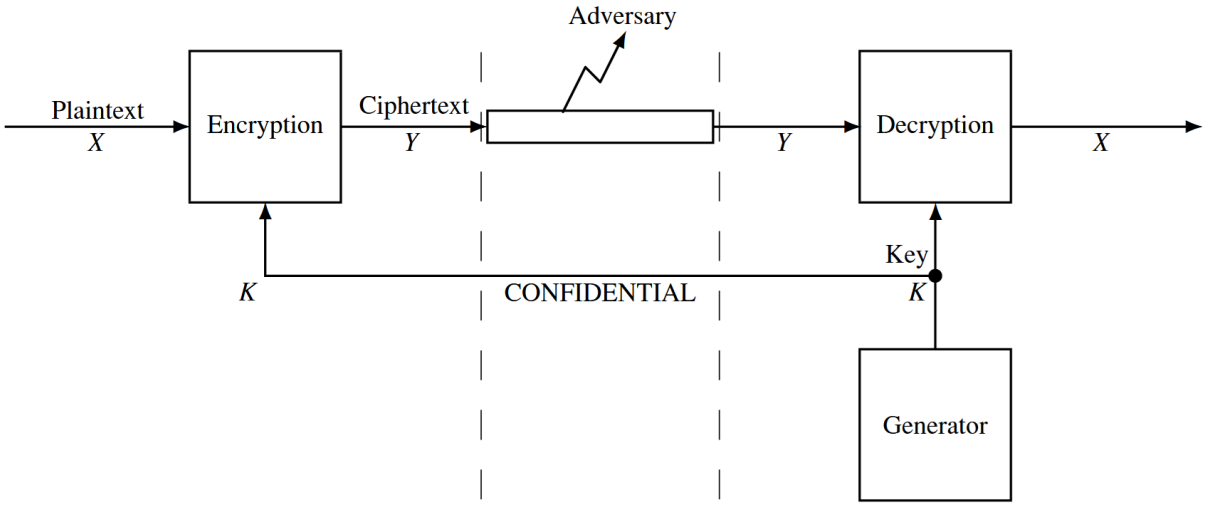


Figura 2.2: Cifrado Simétrico

una manera confidencial, mientras que el atacante intenta obtener la información cifrada.

El **estándar avanzado de cifrado (AES, por sus siglas en inglés)** es uno de los algoritmos de cifrado simétrico más utilizados en la actualidad. Se ha convertido en un componente esencial de numerosos sistemas y protocolos de seguridad ampliamente desplegados, tales como IPsec, TLS, SSH, y el cifrado de redes Wi-Fi mediante el estándar IEEE 802.11i. Hasta la fecha, no se ha descubierto ningún ataque criptográfico práctico más eficiente que la fuerza bruta, lo que lo convierte en una opción robusta y confiable para proteger información confidencial [8]. AES soporta bloques de datos de 128 bits y admite claves de 128, 192 o 256 bits. Gracias a esta solidez, AES sigue siendo a día de hoy el estándar predominante para el cifrado simétrico en la mayoría de los sistemas modernos.

Por otro lado, los cifrados asimétricos -también conocido como criptosistemas de clave publica- está definido por tres algoritmos, un generador de claves pseudoaleatorios, que produce un par de claves (publica K_p, y una privada K_S); un algoritmo de cifrado, que puede ser probabilístico y produce un cifrado Y a partir del mensaje X y la clave publica; y un algoritmo de descifrado, que es determinista y recupera el mensaje original X a partir del cifrado Y y la clave secreta.

Figura 2.3 se muestra un modelo de sistema asimétrico en el que el texto plano se encripta con una clave publica y descifrada con la llave secreta. Es importante destacar, que aun con el acceso a la clave publica, debe ser computacionalmente intratable recuperar el mensaje sin consultar la llave privada.

Aunque los algoritmos clásicos, han demostrado ser eficientes durante décadas, el desarrollo de la computación cuántica plantea nuevas amenazas que comprometen su seguridad. Lo que ha impulsado las investigaciones en nuevos esquemas resistentes a ataques cuánticos.

Para entender como la computación cuántica puede romper los esquemas criptográficos clásicos, debemos entender sus dos principales algoritmos: **Shor** y **Grover**. El algoritmo de Shor, desarrollado por Peter Shor en 1994 [**?**], demuestra que los ordenadores cuánticos pueden

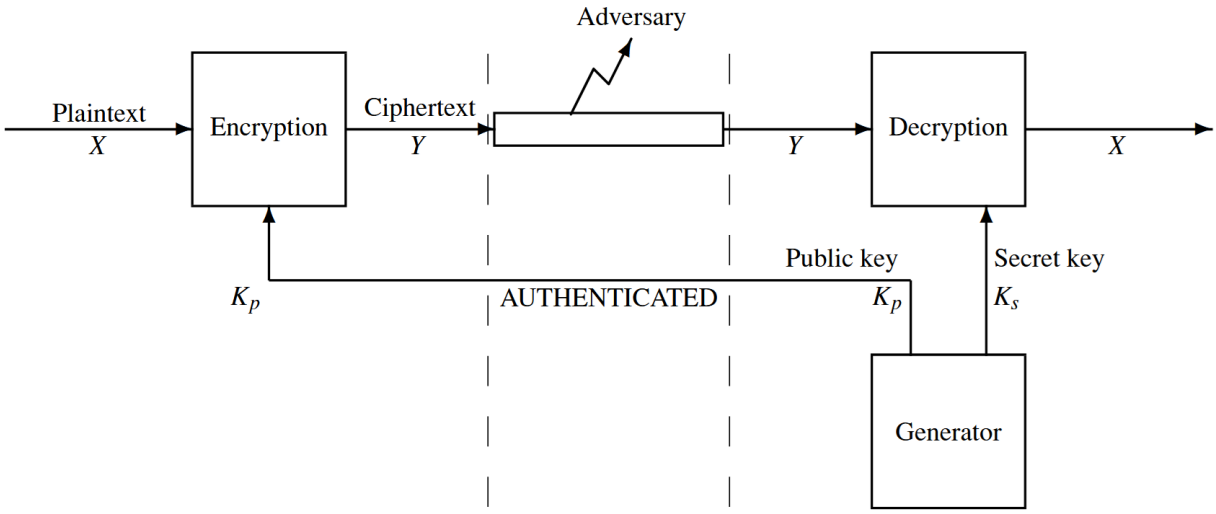


Figura 2.3: Cifrado Asimétrico

resolver dos problemas matemáticos de especial complejidad, que los ordenadores clásicos no pueden resolver en un tiempo razonable: la factirización de números enteros y el calculo de logaritmos discretos. Ambos problemas constituyen la base de la seguridad de sistemas criptográficos de clave pública. Shor muestra que se pueden resolver mediante algoritmos cuánticos cuyo tiempo de ejecución crece de manera polinómica con el tamaño de entrada.

Este resultado es relevante para RSA. La seguridad de RSA depende de la dificultad de obtener los facotres primos de un número entero grande. El algoritmo de Shor transforma el problema de factorización en otro problema, denominado *Busqueda del orden*. Para un entero x coprimo con N, se busca el menor entero positivo r que satisface:

$$x^r \equiv 1 \mod N \tag{2.1}$$

Una vez que se encuentra el periodo r, es posible calcular factores de N utilizando algoritmos clásicos como el maximo común divisor (**MCD**).

La ventaja cuántica de Shor se encuentra en un registro que puede representar una *superposición de muchos valores simultáneamente*. Shor utiliza esta propiedad junto con operaciones de exponenciación modular y la *Transformada Cuántica de Fourier (**QFT**)*. La QFT permite extraer información sobre el periodo existente en los estados cuánticos, que constituye el núcleo del algoritmo.

Además, Shor desarrolla el algoritmo para resolver el problema del logaritmo discreto. Dado un generador g y un valor x, se busca el entero r que satisface:

$$g^r \equiv x \mod p \tag{2.2}$$

El algoritmo muestra también que se puede resolver mediante la exponenciación modular y la

QFT, obteniendo un algoritmo eficiente en un ordenador cuántico. Esta segunda aportación es tan relevante como la primera: implica que los sistemas cuya seguridad depende del logaritmo discreto, como el Diffie-Hellman y, por extensión, los sistemas basados en curvas elípticas, también son vulnerables.

Shor no demuestra que un ordenador cuántico es "más rapido". Demuestra que ciertos problemas matemáticos, en los que se basan los sistemas criptográficos de clave pública clásica cambian de complejidad cuando se dispone de un ordenador cuántico.

Por otro lado, Lov K. Grover, en 1996 [**?**], desarrolló un algoritmo que permite buscar un elemento en una lista desordenada de N elementos. Si no existe ninguna estructura que permita saber dónde se encuentra el elemento buscado, un algoritmo clásico debe revisar cada elemento de la lista. Lo que en promedio requiere:

$$\frac{N}{2} \text{ iteraciones} \tag{2.3}$$

En términos de complejidad, el problema requiere:

$$O(N) \text{ iteraciones} \tag{2.4}$$

Por lo tanto, el algoritmo proporciona una *aceleración cuadrática* respecto a los algoritmos clásicos. Grover señala que existe un límite inferior de $\Omega(\sqrt{N})$ para este tipo de búsquedas cuánticas, por lo que el algorimo de Grover es los más eficiente posible.

La idea fundamental del algoritmo de Grover es representar simultáneamente mendiante superposición cuántica todos los posibles estados del espacio de búsqueda. Inicialmente, los N posibles elementos reciben la misma amplitud:

$$\frac{1}{\sqrt{N}} \tag{2.5}$$

De esta manera, ningún elemento tiene inicialmente una probabilidad mayor que los demás de ser observados.

Existe una operación que permite identificar el estado correspondiente a la solución sin medir directamente el sistema. Esta función, se suele describir como *Oráculo*, modifica la fase del estado que cumple la condición de búsqueda:

$$|x\rangle \rightarrow -|x\rangle \tag{2.6}$$

cuando x es la solución, mientras que los demás estados permanecen sin modificar. Después se aplica la *transformación de difusión*, conocida como *Grover diffusion operator*, que modifica las amplitudes de los estados de manera que la amplitud asociada a la solución aumenta, mientras disminuyen las amplitudes de los demás estados.

El proceso tiene una complejidad de:

$$O(\sqrt{N}) \text{ iteraciones} \tag{2.7}$$

Después de las iteraciones, la probabilidad de medir el estado buscado es suficientemente alta como para recuperar la solución.

Supongamos una clave simétrica de n bits. Existen:

$$N = 2^n \text{ posibles claves} \tag{2.8}$$

Por fuerza bruta, a un atacante le tomaria 2^n intentos encontrar la clave correcta. Sin embargo, al aplicar Grover:

$$O(\sqrt{2^n}) = O(2^{\frac{n}{2}}) \text{ iteraciones} \tag{2.9}$$

Es decir, de manera idealizada, la seguridad efectiva frente a búsqueda exhaustiva se reduce aproximadamente a la mitad en términos de bits.

Como se mencioná en [9], la computación cuántica es una inovación disruptiva extremadamente poderosa, que tiene el potencial de romper los esquemas criptográficos de clave publica ampliamente utilizados, como se demostró anteriormente. Mientras que no sabemos cuando los ordenadores cuánticos estarán disponibles, los investigadores y autoridades institucionales están trabajando en soluciones. Como resultado, **NIST**, aun esta en proceso de estandarización de algoritmos criptográficos resistentes a ataques cuánticos.

Debemos hacer la distinción entre criptografía post-cuántica (**PQC**) y criptografía cuántica. PQC se basa en diseñar soluciones criptográficas que se pueden usa en los ordenadores no cuánticos, que creemos que pueden resistir criptoanalisis tanto cuanticos como convencionales. Por otro lado, la criptografía cuántica se basa en soluciones criptográficas que toman ventaja de las propiedades de la física cuántica para proporcionar ciertos servicios de seguridad, como la distribución de claves cuánticas (**QKD**).

En el trabajo de Schöffel et al [10] se define la criptografía basada en códigos (*code-based cryptography*) como una familia de criptografía post-cuántica cuya seguridad se apoya en la dificultad de resolver detrerminados problemas asociados a los códigos correctores de errores. Estos códigos se utilizan originalmente para detectar y corregir errores durante la transmisión de información, pero tambien pueden emplearse para construir sistemas criptográficos cuando el proceso de decodificación resulta computacionalmente difícil para un atacante.

En términos generales, estos esquemas introducen de forma controlada errores en la información codificada. El receptor legítimo dispone de la estructura necesaria para corregirlos, mientras que un atacante debe enfrentrarse a un problema de decodificación mucho más complejo. Sin embargo, las implementaciones basadas en códigos suelen presentar una mayor complejidad computacional y mayores requsitos de memoria.

Dentro de esta familia se encuentra **HQC**, que combina un código decodificable con un código aleatorio de estructura doblemente circulante. Esto permite reducir el tamaño de las claves respecto a otros esquemas basados en códigos.

2.2.2 HQC

HQC (*Hamming Quasi-Cyclic*) es un mecanismo de encapsulación de claves post-cuántico basado en códigos. Su objetivo no es cifrar directamente todos los datos de una aplicación, sino permitir que dos participantes establezcan de forma segura un secreto compartido, que después es capaz de emplearse con criptografía simétrica como AES. La especificación distingue entre **HQC-PKE**, el esquema de cifrado de clave pública subyacente, y **HQC-KEM**, que construye sobre el mecanismo de encapsulación. Según [11].

La seguridad de HQC se basa en el **Quasi-Cyclic Syndrome Decoding Problem** (**QCSD**), un problema de la teoría de códigos. Conceptualmente, el atacante conoce cierta información sobre un código y debe recuperar un vector de errores de pequeño peso de Hamming. Para ciertos parámetros, encontrar ese vector es computacionalmente complicado. La especificación usa variantes más concretas de este problema, como **2-DQCSD-P** y **3-DQCSD-PT**. La estructura *Quasi-Cyclic* permite representar matrices mediante vectores y rotaciones circulantes, reduciendo así el espacio necesario para almacenar claves frente a una representación aleatoria. HQC trabaja fundamentalmente sobre vectores binarios que pueden interpretarse como polinomios en el anillo:

$$R = \frac{\mathbb{F}_2[X]}{(X^n - 1)} \tag{2.10}$$

Es por ello, que muchas de las operaciones de HQC terminan siendo sumas, multiplicaciones y rotaciones sobre grandes vectores binarios.

El peso de Hamming indica cuántas posiciones de un vector binario son distintos de cero. HQC genera diferentes vectores secretos y de error con pesos de Hamming controlados. Estos errores son los que introducen la dificultad criptogáfica, se deben escoger de tal manera que el receptor legítimo siga siendo capaz de corregirlos. HQC incorpora un código auxiliar basado en **Reed-Muller** y **Reed-Solomon**. Su función es permitir recuperar el mensaje a pesar del ruido introducido durante el cifrado.

HQC se entiende mediante tres operaciones:

- KeyGem
- Encaps
- Decaps

Para comenzar el proceso de establecer un sistema seguro, uno de los participantes debe generar una clave pública y privada segura, esto se le conoce como **KeyGen**, para generar estas claves se usan vectores aleatorios de peso reducido y estructura cuasi-cíclica de HQC.

Una vez que la clave pública se hace llegar al segundo participantge, este genera un valor aleatorio, construyendo lo que se conoce como **Ciphertext** empleando la clave pública e introduce los vectores de error adicionales, este proceso se conoce como **Encaps**. En este proceso se obtiene también el *secreto compartido*.

Finalmente, al hacerle llegar el *Ciphertext* al primer paticipante, comienza un proceso de

decodificación (**Decaps**) del esquema en el que se emplea la clave privada para recuperar la información necesaria y reconstruir exactamente el mismo secreto compartido.

El resultado de este proceso resulta en que ambos extremos terminan obteniendo:

$$K_A = K_B \tag{2.11}$$

Después de obtener el secreto, puede utilizarse para derivar claves simétricas.

Actualmente, HQC proporciona tres conjuntos de seguridad:

- HQC-1,
- HQC-3,
- HQC-5

correspondientes respectivamente a los niveles de seguridad NIST 1, 3 y 5. Estos conjuntos controlan aspectos como n, n_1, n_2, k, ω, ω_r, ω_e.

De manera simplificada:

- n determina el tamaño principal de los vectores utilizados.
- n_1 y n_2 están relacionados con el código concatenado de corrección de errores.
- k determina el tamaño de la información codificada.
- ω, ω_r y ω_e determinan los pesos de Hamming de distintos vectores secretos o de error.

A medida que aumenta el nivel de seguridad, estos valores aumentan, por lo que aumenta también el tamaño de *las claves*, *el ciphertext* y *el coste computacional.*

Por lo que hemos visto, HQC presenta un coste computacional significativo, la implementación en C de referencia necesita, para **HQC-1**, aproximadamente: 4,577 x 10^6 ciclos para KeyGen, 9,116 x 10^6 ciclos para Encaps, y 13,918 x 10^6 ciclos para Decaps. Sin embargo, los benchmarks se realizan sobre un **Intel Core i7-11850H**, por lo que no llegan a representar el comportamiento en microcontroladores.

En arquitecturas ARM Cortex-M4, el coste de HQC está condicionado por las operaciones de multiplicación de polinomios binarios. Por ello, la optimización de estas operaciones permite reducir significativamente el número de ciclos necesarios para KeyGen, Encaps y Decaps, como se menciona en *Optimized implementation of HQC on Cortex-M4*

2.2.3 Sistemas embebidos e IoT

El mundo esta dirigido por sistemas embebidos, cubren aplicaciones especificas de todos los dispositivos que usamos diariamente, desde pequeños termostatos, hasta marcapasos. Como se describe en *Making Embedded Systems* [12], es un sistema diseñado específicamente para una aplicación concreta. A diferencia de un ordenador de proposito general, la misión de un sistema embebido es limitada a una tarea, por lo tanto, el hardware requerido siele estar ajustado a esa función. Esto implica restricciones frecuentes de velocidad de CPU, cantidad

de memoria, y que periféricos estan disponibles.

La caracteristica principal de un sistema embebido, como se mencionó, es la limitacion de recursos:

- Memoria RAM
- Espacio de código (memoria ROM)
- Ciclos del procesador
- Consumo energético
- Periféricos disponibles

Estos recursos limitados se relacionan entre si, el aumento de espacio que ocupa el código puede, en determinados casos, permitir una reducción del tiempo de ejecución, por lo que, existe un compromiso entre los recursos disponibles. Por lo que su correcta optimización, es un reto de diseño.

Para un correcto diseño de un sistema embebido, se debe considerar algunos principios como: modularidad, encapsulación, pruebas y documentación. Separar el sitemas en módulos poco acoplados puede facilitar la modificación de algunas partes sin afectar al resto.

Los sistemas embebidos estan estrechamente relacionados con los dispositivos IoT (*Internet of Things*). Un dispositivo IoT, se puede definir como, cualquier objeto con sensores que pueda recopilar, enviar y recibir información a través de internet, sin que un humano intervenga.

Arshdeep Bahga, en *Internet Of Things: A Hands-On Approach* , lo define como, una infraestructura dinámica y global, basada en protocolos de comunicación interoperables, en los que los objetos físicos se integran en la red de información y pueden intercambiar datos.

Las "Cosas" en IoT se suele referir a dispositivos (o sistemas embebidos), que pueden proveer capacidades de, teledetección, teleactuación y monitoreo. Esta información recolectada se puede tratar localmente, o enviarla a servidores basados en la nube, o realizar alguna tarea local con los datos obtenidos, por lo tanto, un dispositvo IoT consiste en diferentes interfaces de conexión con otros dispositivos, cableada o inalambricamente.

En la mayoria de entornos IoT, las comunicaciónes inalambricas toman una mayor importancia, ya que permite desplegar una red de dispositivos sin la necesidad de una gran infraestructura física de conexión. Sin embargo, el uso de medios inalambricos introduce restricciones como el alcance, ancho de banda, o consumo energético. De estas limitaciones, nace la necesidad de estudiar los mecanismos empleados para transmitir dicha información.

El modelo OSI (*Open Systems Interconnection*) nos explica como funcionan las comunicaciones de los dispositivos:

Como se muestra en la Figura 2.4, el modelo OSI organiza las comunicaciones por capas, cada una teniendo una tarea especifica, desde la transmisión fisica de los datos, hasta los servicios utilizados por las aplicaciones. Aunque los dispositivos IoT no siempre iguen extrictamente las capas, el modelo nos resulta útil para descrribir la función de los distintos protocolos de comunicación. En este trabajo nos centraremos en las capas física y de enlace, que son



Figura 2.4: Modelo OSI.

las encargadas de la transmisión inalámbrica; la capa de red, que es la responsable del direccionamiento; la capa de transporte, donde se empleta UDP; la capa de aplicación es donde integramos los mensajes de HQC.

La capa de enlace determina como se envia los datos fisicamente a traves de la red. Su ambito es la conexión de red local a la que está conectado el host. Los host se encuentran en el mismo enlace donde intercambian paquetes de datos. La capa de enlace determina como el dispositivo codifica y transmite los paquetes a través del medio al que esta conectado.

- 802.11 - WiFi: Es un conjunto de estándaers de comunicación para redes de area local inalámbricas (WLAN), describe detalladamente la capa de enlace. Por ejemplo, el estándar 802.11a opera en la banda de 5Ghz.

La capa de red es la responsable del envío de datagramas IP (*Internet Protocol*) desde el origen hasta el destinatario. Esta capa se encarga del direccionamiento del host y el enrutamiento de los paquetes. La identificacion de los host se realiza a través de esquemas jerárquicos de direccionamiento IP, como IPv4 o IPv6.

- IPv4: Es el protocolo de internet versión 4, es el más extendido que se usa para identificar dispositivos de una red mediante el esquema jerárquico, usando un esquema de direcciones de 32 bits.
- IPv6: Es la versión 6 del protocolo de internet más reciente, utilizando un esquema jerárquico de 128 bits.

La capa de transporte proporciona la capacidad de transferir los mensajes independientemente de la red subyacente, la capacidad de transferir estos mensajes se puede configurar en las conexiones, ya sea mediante procedimientos de establecimiento de conexión (TCP) o sin ellos, ni acuses de recibo (UDP). Esta capa proporciona funciones como el control de errores, fragmentacion, control de flujo y congestión.

- TCP: Protocolo de control de transmisión, es el protocolo más utilizado, TCP garantiza una transmisión fiable de paquetes en el orden correcto.
- UDP: A diferencia de TCP que requiere una configuración inicial, UDP (*User Datagram Protocol*) es un potocolo sin conexión. Es útil para aplicaciones en las que el tiempo es crítico. UDP no garantiza la entrega ni el orden de los mensajes.

Es en este contexto donde intervienen las redes de sensores inalámbricos (WSN, por sus siglas en inglés), una red de sensores inalambricas consta de nodos finales, enrutadores, y coordinadores. Los nodos finales también pueden actuar como enrutadores, estos ultimos se encargan de distribuir los datos de los nodos finales hasta el coordinador. El coordinador recopila estos datos y actua sobre ellos, también tiene la capacidad de conectar la WSN a internet. Algunos casos en los que se encuentran WSN son:

- Los sistemas de monitorización meteorológica utilizan WSN en la que los nodos recogen datos de temperatura, humedad y otros parámetros que luego son analizados.
- Los sistemas de vigilancia también utilizan WSN para recopilar datos como los de movimiento.

Sin embargo, debemos entender que el medio no es ideal, como lo explica Theodore S. en, *Wireless communications: principles and practice* [13]. Existen limitaciones en el rendimiento de las comunicaciones inalámbricas, la trayectoria de transmisión entre el que transmite y el receptor puede variar de una línea de visión directa hasta una gravemente obstruida.

El modelo de propagación en espacio libre se utiliza para predecir la intensidad de la señal cuando el transmisor y el receptor tienen una linea de visión directa, sin obstaculos. Al igual que la mayoria de modelos, este predice que la potencia recibida deace en función de la distancia que separa el transmisor del receptor, para una determinada potencia.

La perdida de trayectoria, representa la atenuación de la señal como una magnitud positiva medida en dB, es la diferencia entre la potencia efectiva transmitida y la potencia recibida, puede incluir o no la ganancia de las antenas. La pérdida de trayectoria para el modelo de espacio libre, incluyendo las ganancias de las antenas se define como:

$$PL(\text{dB}) = 10\log\frac{P_t}{P_r} = -10\log\left[\frac{G_t G_r \lambda^2}{(4\pi)^2 d^2}\right] \tag{2.12}$$

En un entorno real, como una WSN, se tienen diferentes mecanismos por los que se puede tener una peor comunicación. Estos mecanismos son la reflexión, difracción y dispersión. La relexión se produce cuando una onda electromagnética incide sobre un objeto cuyas dimenciones son muy grandes en comparación con la longitud de onda propagada, esta se produce sobre la superficie de la tierra, edificios, entre otros. La difracción se produce cuando la trayectoria de

la onda de radio entre el transmisor y el receptor se obstruye por una superficie que presenta irregularidades. La dispersión se produce cuando el medio por el que viaja la onda está formado por objetos cuyas dimensiones son pequeñas, y cuando el número de obstáculos por unidad de volumen es elevado. Se producen por superficies rugosas, objetos u otras irregularidades en el canal. Debido a estos mecanismos surge un fenomeno llamado **Multipath**.

La consecuencia directa del *Multipath* es el desvanecimiento a pequeña escala. Existen tres efectos importantes:

- Cambios rápidos en la intensidad de la señal en un intervalo reducido de tiempo.
- Modularización aleatoria de la fracuencia debido al desplazamiento Doppler.
- Dispersión temporal causada por el retardo introducido por el multipath.

En una red multi-hop, un paquete puede atravesar varios nodos intermedios antes de alcanzar su destino. Cada salto requiere una nueva transmisión sobre el medio inalámbrico, aumentando el uso del canal y pudiendo incrementar la latencia extremo a extremo. Asimismo, cuando varios flujos convergen sobre determinados nodos intermedios, puede producirse una mayor concentración de tráfico y aumentar la probabilidad de contención, colisiones o retransmisiones.

Por lo que se ha estudiado, las comunicaciones inalambricas estan lejos de ser perfectas, y se ha visto como los paquetes pueden verse afectados tanto como por el entorno, causando una atenuación y desvanecimiento de la señal, como por compartir el canal de comuncaciones con otros dispositivos puede introducir retrasos en los paquetes y retransmisiones no deseadas.

2.2.4 Criptografía post-cuántica en sistemas embebidos e IoT

Haciendo una revisión de "Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization", de Tao Liu, Gowri Ramachandran, y Raja Jurdak, se obtiene una conclusión clara: *PQC puede ser una alternativa viable en dispositivos IoT con recursos limitados, pero falta una metodología estandarizada para evaluar su coste* [14].

Como se ha revisado anteriormente, los dispositivos IoT y sistemas embebidos, son dispositivos de recursos limitados, CPU limitada, poca RAM, ancho de banda reducido. Los algoritmos PQC suelen manejar mayor coste computacional, memoria, tamaño de transmisión y consumo energético para estos dispositivos. Es por ello que se concentran los esfuerzos en las optimizaciones de los algoritmos en este tipo de dispositivos.

Mayoritariamente, los analisis realizados sobre el rendimiento de los algoritmos PQC se centran en el tiempo de ejecución o el número de ciclos. Sin embargo, Liu et al. señalan que estas métricas no son suficientes para caracterizar su impacto en redes IoT. Hay que destacar también la importancia del uso de memoria, el tamaño del codigo, y el tamaño de los datos transmitidos.

Además de lo explicado en el paper anterior, se debe tener en cuenta que la incorporación de PQC introduce nuevas restricciones relacionadas con la comunicación. El mayor problema es el tamaño de las claves, mensajes cifrados y firmas, que incrementan el volumen de datos transmitidos. A pesar de estas limitaciones, los esquemas PQC pueden resultar viables en

dispositivos IoT, siendo su mayor limitante el rendimiento, que depende tanto del algoritmo utilizado, como de la plataforma de ejecución. Como consecuencia, las propuestas actuales se centran en reducir el tiempo de ejecución y el uso de memoria.

2.3 Trabajos Relacionados

2.3.1 Code-based Cryptography in IoT: A HW/SW Co-Design of HQC

En el trabajo de Schöffel et al. "*Code-based Cryptography in IoT: A HW/SW Co-Design of HQC*", se parte del problema de que los algoritmos post-cuánticos, como HQC, suelen requerir más computo y memoria. Es por ello, que las implementaciones en dispositivos IoT suelen ser menos estudiadas en este ambito.

Para empezar, los autores realizaron la implementacion del perfil de HQC-128 sobre un procesador RISC-V de 32 bits, con el objetivo de cuales son las operaciones que más memoriua y ciclos consumen. Seguidamente, se decide que optimización es mejor para cada cuello de botella.

Muestran que los mayores costes de HQC aparecen principalmene en tres operaciones:

- Operaciones aritméticas sobre polinomios.
- Operaciones criptográficas de SHAKE/Keccak.
- Operaciones de memoria.

En la implementaciópn de referencia, las operaciones de memoria representan un 37 % del tiempo en KeyGen, 36.6 % en Encaps y 36 % en Decaps, mientras que SHAKE supone un 27 % y un 36 % y la aritmetica polinómica alrededor de un 25 %. Por lo que se puede concluir que el coste de HQC no proviene únicamente de "hacer criptografía", sino también de las gestiones de datos.

En el paper se proponen varias optimizaciones, como el optimizado el código C eliminando copias innecesarias, trabajando con punteros y usando DMA para acelerar memcpy y memset.

Las optimizaciones de software, junnto con el uso de DMA permiten reducir el tiempo de ejecución entre un 36 % y un 49 % respecto a la implementación de referencia. Al combinar las optimizaciones de software y hardware, se consiguen unas reducciones cercanas al 99 % en el númeo de ciclos necesarias para las operaciones de HQC. Estos resultados muestran que HQC es una alternativa viable para dispositivos IoT de recursos limitados; también muestran que la implementación de referencia de HQC no es adecuada para este tipo de dispositivos, y que se requieren optimizaciones específicas para reducir el coste computacional y de memoria.

2.3.2 PQM4: Testing and Benchmarking NIST PQC on ARM Cortex-M4

El paper de Kannwischer et al. "*PQM4: Testing and Benchmarking NIST PQC on ARM Cortex-M4*" [**?**] Trata de construir una plataforma común de pruebas y benchmarking para comprobar que algoritmos PQC se pueden ejecutar en un microcontrolador. Su objetivo principal es evaluar la viabilidad de los candidatos del proceso de estandarización post-cuántica del NIST sobre un ARM Cortex-M4, utilizando una placa de desarrollo STM32F4DISCOVERY, con 196 KiB de RAN y 1 MiB de memoria flash.

El framwork PQM4 proporciona distintas implementaciones de algoritmos PQC, asi como sus evaluaciones. Además, tienen diferentes distinciones segun el nivel de implementación: Código de referencia, implementaciones C optimizadas e implementaciones específicamente optimizadas para Cortex-M4.

Este trabajo mide principalmente:

- Número de ciclos de CPU.
- Consumo de memoria de pila.
- Tamaño del código.
- Tiempo empleado en Hashing.
- Corrección funcional de los KEM.

Para los KEM, se hace la distinción entre KeyGen, Encaps y Decaps. Se intenta homogeneizar la comparacion, haciendo que todos los algorritmos utilicen implementaciones como AES, SHA2, SHA3 y SHAKE.

Los resultados son interesantes, mostrando que no todas las implementaciones de PQC son adecuados para dispositivos IoT y sistemas embebidos. Es de especial relevancia que en algunos casos el problema era el tamaño de las claves, por ejemplo, Classic McEliece, tiene claves públicas comprendidas entre 255 KiB y 1326 KiB, superiores a la memoria disponible en el microcontrolador.

Los autores destacan que el rendimiento de los algoritmos depende considerablemente del grado de optimizaciópn de la implementación, es por ello que los resultados no deben interpretarse como comparación directa entre los algoritmos.

En el caso de HQC, no fue posible realizar su evaluación, debido a que las implementaciones disponibles al momento dependían de bibliotecas externas dificultando su integración en la plataforma.

2.3.3 Optimized Implementation of HQC on Cortex-M4

El artículo de DongCheon Kim et al. "*Optimized Implementation of HQC on Cortex-M4*" NO OLVIDAR CITAR se parte de la premisa de que HQC tiene un coste importante en plataformas limitadas. Su objetivo es optimizar HQC para que pueda ser ejecutado de forma más eficiente en un microcontrolador ARM Cortex-M4.

Para el momento de escritura de este artículo, ya existían optimizaciones para HQC en Cortex-M4, basadas en aprovechar el hecho de que algunos vectores secretos son spares; se almacenan en las posiciones de los bits a 1 y luego se realizan desplazamientos y XOR sobre ellas. Esto introduce un problema, ya que, las operaciones realizadas dependen del secreto, y el tiempo de ejecución puede variar, creando asi una potencial amenaza de canal lateral.

Los autores investigan primero la implementación de PQClean incluida en PQM4, descubriendo que la multiplicación de polinomios binarios domina el tiempo de ejecución; En HQC_256, vect_mul representa el 97 % del coste de Key Generation, en encapsulación representa el 98 % y en desencapsulación representa el 96 %. Es por ello que se enfocan en optimizar esta operación.

Se proponen diferentes optimizaciones de algoritmos de multiplicación de polinomios, principalmente Karatsuba y Toom-Cook, seleccionando una configuración diferente según el nivel de seguridad de HQC. Además, se optimizan otras operaciones de bajo nivel teniendo en cuenta las características del Cortex-M4.

Las evaluaciones se realizan utilizando el framework PQM4, sobre una placa NUCLEO-L4R5ZI, con un procesador ARM Cortex-M4 a 20MHz. Las metricas utilizadas son el número de ciclos de reloj y el uso de la memoria de pila. Los resultados muestran mejoras entre un 1042 % y 1348 % frente a la implementación PQClean, dependiendo de la operación y el nivel de seguridad. Frente a la implementación de referencia, previamente optimizada, se obtienen mejoras comprendidas aproximadamente entre un 35 % y un 46 %.

Este artículo demuestra que el coste computacional de HQC se puede reducir significativamente mediante optimizaciones adaptadas a arquitecturas ARM Cortex-M4, manteniendo un uso de memoria de pila similar a la de la implementación de referencia. Sin embargo, los estudios se centran en el rendimiento computacional, sin llegar a medir su impacto derivado de su integración y transmisión.

2.3.4 A Comparative Study of Post-Quantum Cryptosystems for Internet-of-Things Applications

En el artículo de Septien-Hernandez et al. "*A Comparative Study of Post-Quantum Cryptosystems for Internet-of-Things Applications*" NO OLVIDAR CITAR, se realiza un estudio comparativo de se estudia la viabilidad de integrar algoritmos PQC eden dispositivos IoT con recursos limitados. Se parte de la premisa de que la criptografía post-cuántica afectara también este tipo de dispositivos, y debido a sus limitaciones, es necesario analizar que algoritmo es el más adecuado.

HQC aparece entre los candidatos considerados inicialmente dentro del proceso de estandarización del NIST. Sin embargo, los autores centran el estudio experimental en algoritmos basados en retículas, ya que, de acuerdo con la literatura considerada, estos presentan características más adecuadas para dispositivos con recursos limitados. Los algoritmos seleccionados para el análisis fueron LightSaber, Kyber512, NTRUhps2048509, NTRULPr653 y FrodoKEM640.

Los analisis se realizaron sobre una arquitectura IoT compuesta por una red inalámbrica de sensores, un *getaway* y un servidor en la nube. Los nodos sensores utilizan un Arduino

Nano con un microcontrolador ATmega328P, un sensor DHT22 y un módulo de comunicación RFM69HCW. Como *gateway* se utiliza una Raspberry Pi 3B+, mientras que a nivel software se emplean TLS 1.3, MQTT y una versión modificada de OpenSSL con soporte para algoritmos post-cuánticos.

Para analizar el rendimiento de los algoritmos se consideran principalmente tres métricas: uso de CPU, memoria RAM y tráfico de red. Para el tráfico de red se considera el número de paquetes transmitidos, el número total de bytes y la duración de la conexión.

Los resultados muestran diferencias importantes entre los diferentes mecanismos. En consumo de memoria LightSaber presenta el mejor resultado, con un máximo aproximado de 994 bytes, mientras que FrodoKEM640 alcanza aproximadamente 921 kB. En tiempo de ejecución Kyber512 obtiene el mejor comportamiento, con aproximadamente 204 ms para el conjunto de operaciones de generación de claves, encapsulación y decapsulación, seguido de LightSaber con unos 255 ms, En términos generales, las curvas elípticas requieren menos bytes transmitidos que los mecanismos post-cuánticos, aunque los resultados indican que estos últimos pueden integrarse en sistemas IoT actuales con un rendimiento razonable.

Este trabajo resulta interesante, debido a que no limita la evaluación al coste computacional, sino que también considera el impacto sobre las comunicaciones. No obstante, HQC no forma parte de los algoritmos evaluados, por lo que queda abierta la necesidad de estudiar su comportamiento en escenarios IoT reales.

Capítulo 3

Diseño e implementación del sistema

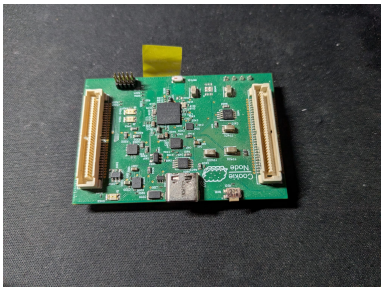
Tras haber realizado la revisión de los fundamenteos de la criptografía post-cuántica y revisar los principales trabajos relacionados con la integración en sistemas embebidos, se observa que los esfuerzos existentes se centran en evaluar el rendimiento de los algoritmos sobre las plataformas con recursos limitados, mediante metricas como el tiempo de ejecución, el uso de memoria o el consumno de recursos.

En este capítulo discutiremos el diseño e implementación de la plataforma experimental. Se presentara la arquitectura del sistema, la integración de HQC, el mecanismo empleado para su intercambio y transporte a través de una red inalambrica, las topologías implementadas y el sistema de adquisición de datos para evaluar su comportamiento.

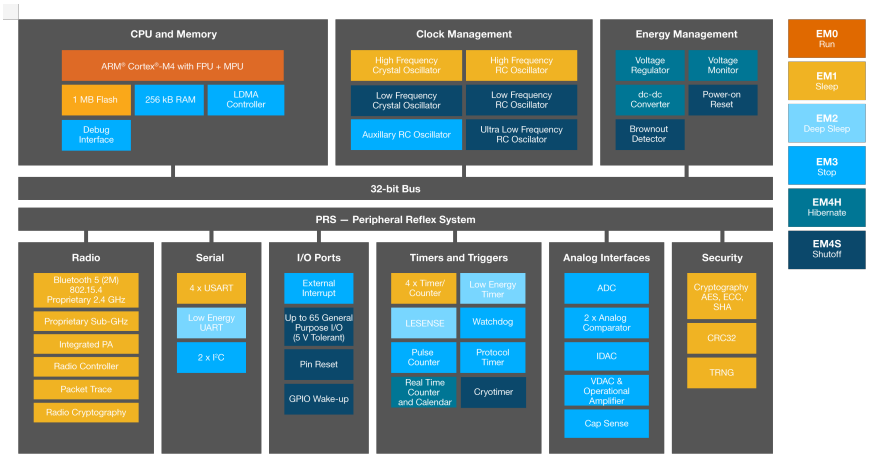
3.1 Plataforma experimental

En primer lugar, debemos presentar la plataforma experimental utilizada para la integración de HQC. Esta se compone de un nodo embebido, que actúa como el dispositivo final, en el que se recolectan las métricas, se implementa el protocolo de comunicaicón, adquisición de datos ambientales, entre otros. El nodo lleva el nombre de *Cookie*, y fue desarrollada en el departamento electronica industrial de la universidad politécnica de Madrid. Se trata de una plataforma de bajo consumo, que incorpora un SoC inalámbrico mutiprotocolo de alto rendimiento fabricado por Silicon Labs, el **EFR32MG21** [**?**], que integra un microcontrolador ARM Cortex-M4 de 32 bits.

El núcleo ARM Cortex-M4, es una arquitectura ampliamente utilizada en sistemas embebidos, debido a que ofrece un gran equilibrio entre rendimiento y consumo de energia. Las características principales de este microcontrolador resultan relevantes, ya que las operaciones criptográficas deben ejecutarse sobre el nodo, compartiendo los recursos disponibles como la pila de comunicaciones, el sistema de adquisición de datos y el resto de funcionalidades. El SoC integra una memoria Flash de hasta 1MB y 256 KB de memoria RAM. El SoC cuenta con un módulo RF integrado y configurable para las comunicaciones inalámbricas. Dependiendo de la variante, permite operar tanto en la banda ISM de 2.4 GHz como en la banda Sub-GHz. Las comunicaciones empleadas en este trabajo se realizan en la banda de 2.4 GHz.



(a) Nodo embebido Cookie.



(b) Diagrama de bloques funcional del SoC inalámbrico EFR32MG21.

Figura 3.1: Componentes de hardware utilizados en el proyecto.

Por otro lado, el nodo embebido cuenta con un sensor de temperatura y humedad frabicado por Silicon Labs, *Si7021* [**?**], que permite la adquisición de datos ambientales. El sensor incorpora un convertidor analógico-digital, que permite una comunicación mediante una interfaz I2C. El sensor cuenta con un calibrado de fábrica y presenta una precisión máxiama de $\pm 3\,\%RH$ para humedad relativa y $\pm 0,4ŀC$ para temperatura.

En la plataforma experimental, las medidas obtenidas por el sensor Si7021 se utilizan para genera un trafico de datos periodicos en la red empleada en el experimento. De esta forma, el envio de temperatura y humedad constituye un flujo de datos que permite evaluar el rendimiento de la red.

3.2 Arquitectura del sistema

Una vez presentada la plataforma experimental, se describe la arquitectura general del sistema. Esta se compone por dos tipos de nodos: cliente y servidor. Los nodos clientes se encargan de adquirir los datos ambientales y transmitirlos al servidor. Por otro lado, el nodo servidor se encarga de recibir y almacenar la información, asi como de gestionar las sesiones asociadas a cada nodo, coordinar el inicio y finalización de las pruebas y enviar instrucciones necesarias a los clientes.

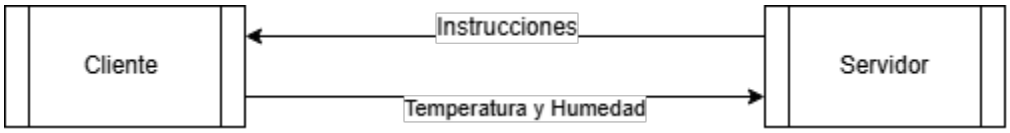
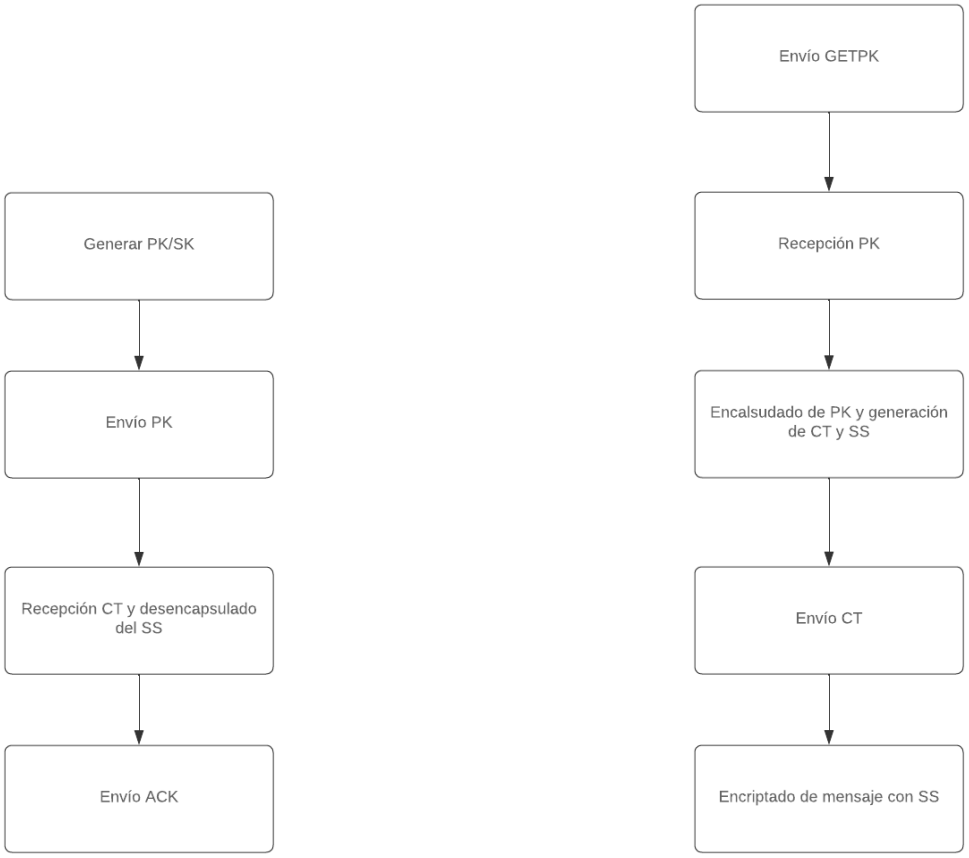


Figura 3.2: Arquitectura general del sistema.

En la Figura 3.3a se muestra el flujo principal de HQC del servidor, en cuanto el nodo servidor se inicia, genera el par de claves pública y privada, y queda a la espera de la llegada de un mensaje de inicio de sesión por parte del cliente, este mensaje se conoce como *GETPK*. Una vez recibido, el servidor responde con la clave pública. Cuando el servidor recibe el CT, ejecuta el proceso de desencapsulación utilizando su clave privada y obtiene el mismo secreto compartido SS que se genero en el cliente. A partir de este momento, la sesión queda establecida y el servidor queda a la espera de la llegada de los datos cifrados del cliente.



(a) Arquitectura del servidor.

(b) Arquitectura del cliente.

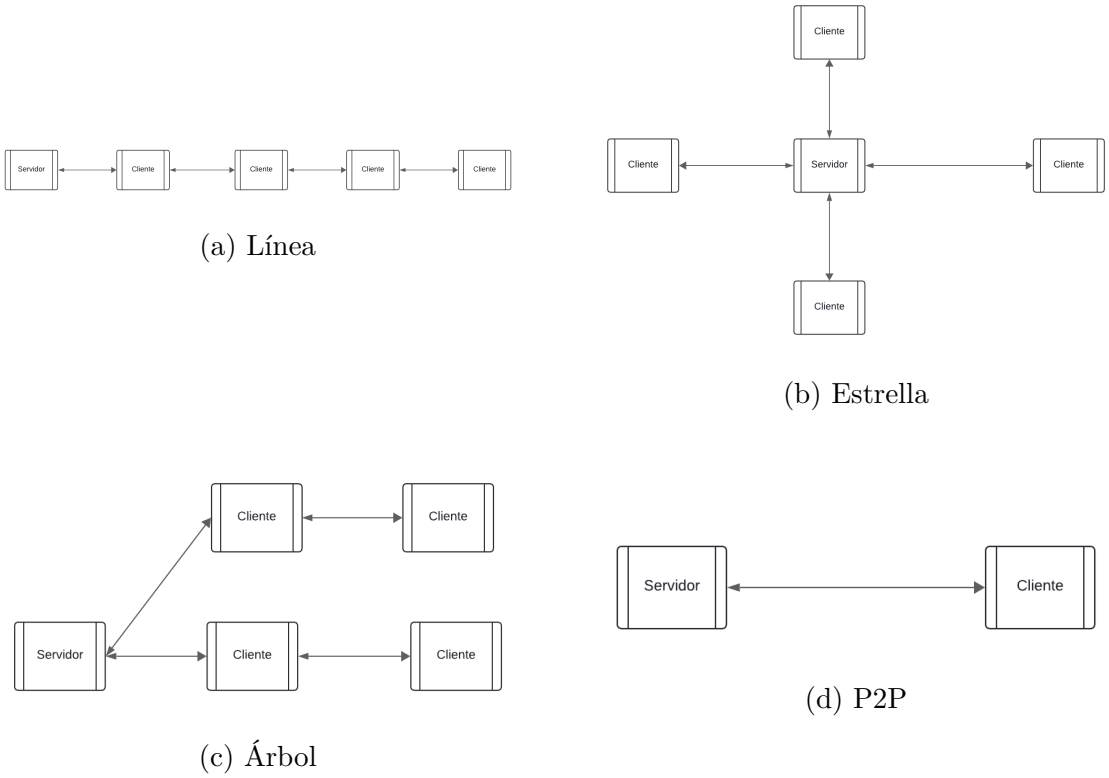
Figura 3.3: Arquitectura del sistema.

Por el lado del cliente, como se muestra en la Figura 3.3b, el flujo de HQC comienza con el envío del mensaje *GETPK* al servidor. Una vez recibida la clave pública del servidor, el cliente ejecuta el proceso de encapsulación utilizando dicha clave, obteniendo como resultado el texto cifrado CT y el secreto compartido SS. Una vez que el cliente recibe la confirmacón de que el servidor ha desencapsulado el CT correctamente, el cliente procede a cifrar los datos obtenidos del sensor y enviarlos al servidor. Una vez establecido el secreto compartido entre el cliente y el servidor, este se asocia a la sesíon del cliente, y es utilizado para proteger las posteriores comunicaciones de la aplicación.

La comunicación entre el cliente y el servidor sobre la pila de red proporcionada por Contiki-NG. Los mensajes son transportados mediante el protocolo UDP sobre IPv6, empleando 6LowPAN para su adaptación al enlace inalámbrico y RPL para establecer las rutas entre los distintos nodos de la red.

La arquitectura descrita en este apartado se mantiene para las diferentes topologías de red implementadas durante los experimentos. Se implementaron cuatro topologías diferentes, estrella, linea, árbol y P2P, modificando el número de nodos y saltos existentes entre los

clientes y el servidor. En la Figura 3.4 se muestran las topologías implementadas.



(a) Línea

(b) Estrella

(c) Árbol

(d) P2P

Figura 3.4: Topologías implementadas.

Las topologías implementadas presentan diferentes características en términos de número de saltos, concentración de tráfico y acceso al medio compartido. Las configuraciones P2P y estrella permiten comunicaciones directas con el servidor, mientras que las topologías en línea y árbol introducen comunicaciones multi-hop. Estas diferencias permiten estudiar posteriormente cómo la estructura de la red condiciona el transporte de los mensajes asociados al establecimiento de sesiones mediante HQC, especialmente aquellos de mayor tamaño, como la clave pública y el CT.

La estructura basica de los mensajes es:

Cabecera del protocolo	**Carga útil**
10 bytes	0–64 bytes

El tamaño maximo de un mensaje fragmentado de la aplicación es de 74 bytes, sin contar con las cabeceras de UDP ni IPv6. El tamaño de la carga útil depende del tipo de mensaje, por ejemplo, para el GETPK, que no necesita transportar información adicional, la carga sera minima, pero PK y CT, contienen fragmentos de las estructuras de HQC.

La cabecera de los mensajes del protocolo contiene información necesaria como versión, tipo de mensaje, identificador del nodo asociado al mensaje, como se muestra en la Tabla 3.1.

Tabla 3.1: Campos y funciones de la cabecera del protocolo.

Campo	Tamaño	Función
`version`	1 byte	Identifica la versión del protocolo.
`msg_type`	1 byte	Indica el tipo de mensaje transmitido.
`node_id`	2 bytes	Identifica al cliente asociado al mensaje.
`msg_id`	1 byte	Identifica el mensaje dentro de la comunicación.
`frag_idx`	1 byte	Indica la posición del fragmento.
`total_frags`	1 byte	Número total de fragmentos del mensaje.
`payload_len`	1 byte	Longitud de los datos contenidos en el fragmento.
`flags`	1 byte	Información adicional de control.
`reserved`	1 byte	Campo reservado.

Se pueden clasificar los tipos de mensajes en tres categorías, mensajes de establecimiento de la sesión (GETPK, PK, CT, ACK), mensajes de control y recuperación (NACK) y mensajes de aplicación e intrumentación (TEMP, METRICS), como se muestra en la Tabla 3.3

Tabla 3.3: Tipos de mensajes, identificadores y funciones en el protocolo.

Tipo	Identificador	Dirección habitual	Función
`PK`	1	Servidor → Cliente	Transportar la clave pública HQC.
`CT`	2	Cliente → Servidor	Transportar el texto cifrado HQC.
`ACK`	3	Servidor → Cliente	Confirmar el procesamiento del CT.
`TEMP`	4	Cliente → Servidor	Transportar medidas ambientales cifradas.
`METRICS`	5	Cliente → Servidor	Enviar información de instrumentación.
`GETPK`	6	Cliente → Servidor	Solicitar la clave pública.
`NACK`	7	Bidireccional	Solicitar fragmentos ausentes.

Existe un caso especial en los mensajes de TEMP, debido a que se deben añadir en la carga útil, los datos de temperatura, humedad y contador, ocupando unos 12 bytes de carga útil.

<table>
<tr><td rowspan="2">Cabecera del protocolo</td><td colspan="3">Carga útil cifrada</td></tr>
<tr><td colspan="3">12 bytes</td></tr>
<tr><td rowspan="2">10 bytes</td><td>counter</td><td>humedad</td><td>temp</td></tr>
<tr><td>4</td><td>4</td><td>4</td></tr>
</table>

3.3 Protocolo de intercambio y transporte de HQC

Como se ha tratado en la sección anterior, el intercambio de HQC se realiza mediante un protocolo de comunicación basadi en mensajes UDP, en el existe un nodo COOKIE que actúa como servidor para resivir todas las peticiones, datos y manejar cada una de las sesiones con los clientes. En la Figura 3.3 se muestra un diagrama de alto nivel del protocolo, en el que se puede observar el flujo de mensajes.

Por otro lado, se implementó un mencanismo de fragmentación en ambos lados de la comunicación, tanto en servidor como en cliente, para fragmentar las claves públicas y textos cifrados, respondiendo a la necesidad de transportar mensajes de mayor tamaño que la carga útil máxima de un paquete UDP. El mecanismo fragmenta los mensajes en fragmentos de 64 bytes de datos, con 10 bytes para la identificación del mensaje, el nodo, la posición del fragmento y otra información de control.

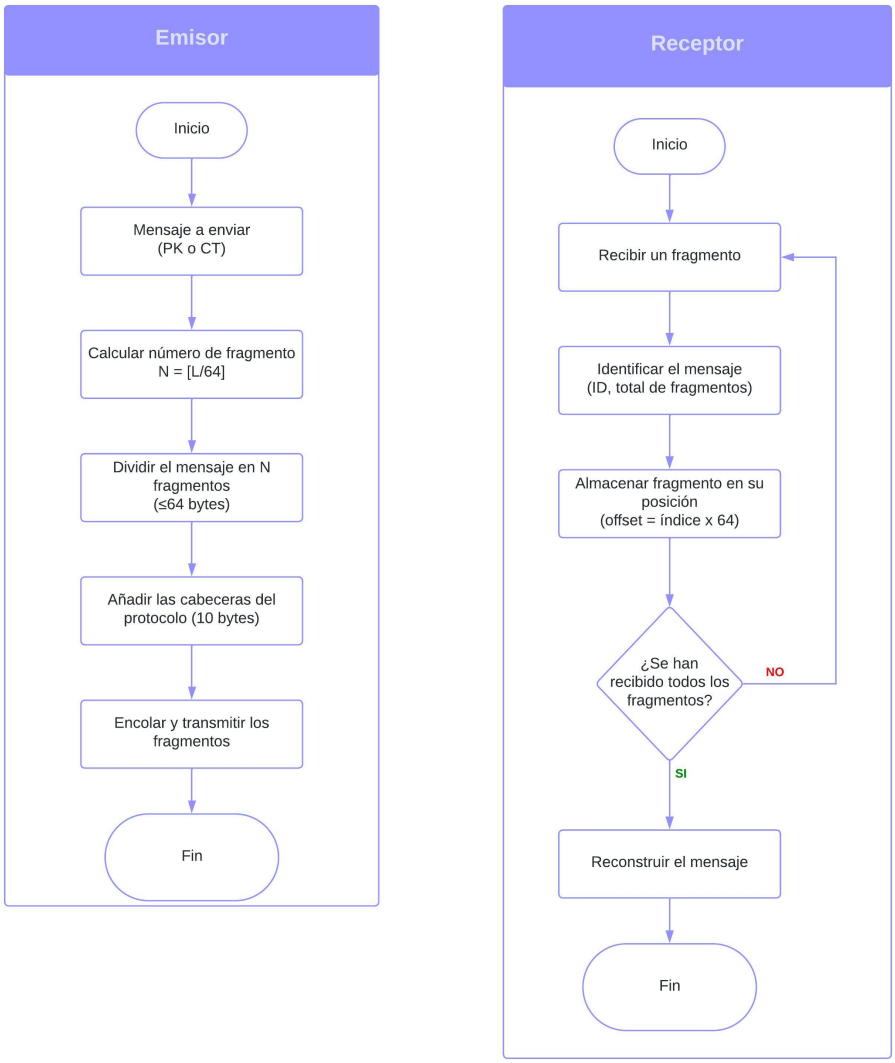


Figura 3.5: Arquitectura de fragmentación.

La Figura 3.5 muestra el flujo de fragmentación y reensamblado de los mensajes. Es impotante acotar, que el mecanismo de fragmentación, resuelve el problema de transportar mensajes de gran tamaño, pero introduce cierto overhead sobre la red, y tampoco garantiza la entrega de los fragmentos. Si un fragmento se pierde, el mensaje completo no puede ser reensamblado, y la sesión ya no seria valida.

Debido a está incidencia, se implementó un mecanismo de retransmisón de mensajes, que permite reintentar el envío de fragmentos especificos, en el caso de que alguno de ellos no sea recibido. También conocido como NACK (negative acknowledgment), permite al receptor identificar que fragmentos no han sido recibidos y los solicita nuevamente. Este mecanismo se implementa en ambos lados de la comunicación.

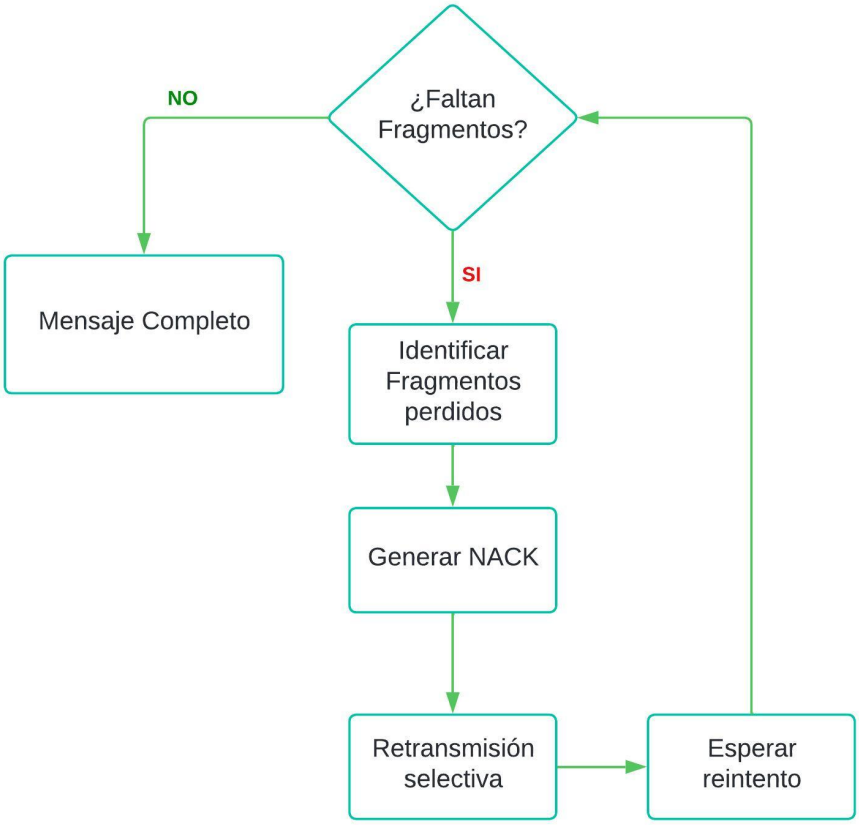


Figura 3.6: Arquitectura de reensamblado de fragmentos.

La Figura 3.6 muestra el flujo de reensamblado y recuperación de fragmentos, en el que se puede observar que el receptor mantiene un buffer de fragmentos, en el que se almacenan los recibidos, y se genera un mensaje NACK en el que se identifican los faltantes. El emisor recibe este mensaje NACK y reintenta el envío de los fragmentos faltantes. Este proceso se repite hasta que todos los fragmentos son recibidos correctamente.

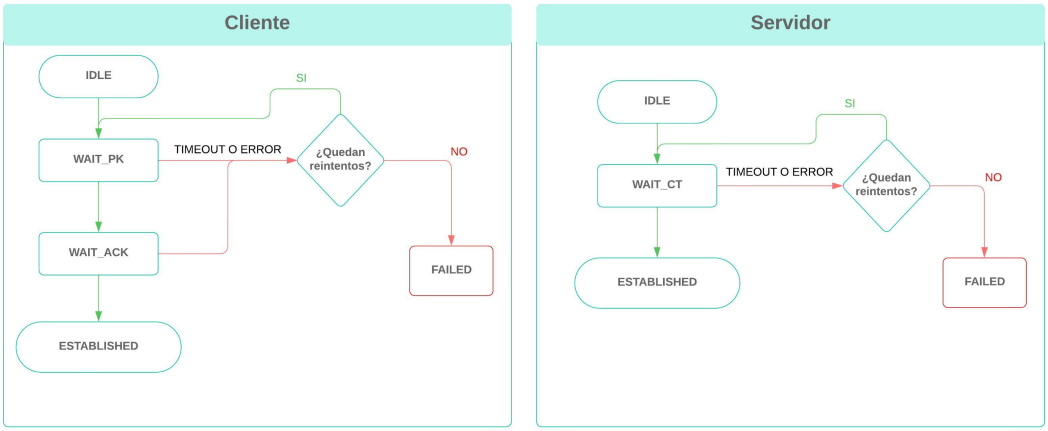


Figura 3.7: Arquitectura de estados.

El diagrama de estados de la Figura 3.7 muestra los estados definidos en el protocolo de comunicación, tanto para el cliente como para el servidor. El servidor se mantiene en estado de espera hasta recibir el mensaje GETPK, momento en el que envía la clave pública generada al inicio del servidor. Al completar el envío se queda a la espera de recibir el CT, y una vez recibido envia una confirmación al cliente, y se queda en el estado de espera de los datos cifrados.

Por otro lado, en una implementación real de las topologías de red, al existir varios nodos conectados al mismo servidor, es posible iniciar varias sesiones de manera simultanea, y el servidor al ser incapaz de atender a todos los clientes al mismo tiempo, en el que cada sesión está en un estado diferente, se debe implementar un mecanismo de prioridades de envío de mensajes. Como se muestra en la Figura 3.8, el servidor mantiene una cola de mensajes, en la que se clasifica el tipo de mensaje a enviar, y se prioriza el envío de los mensajes NACK, ACK y GETPK.

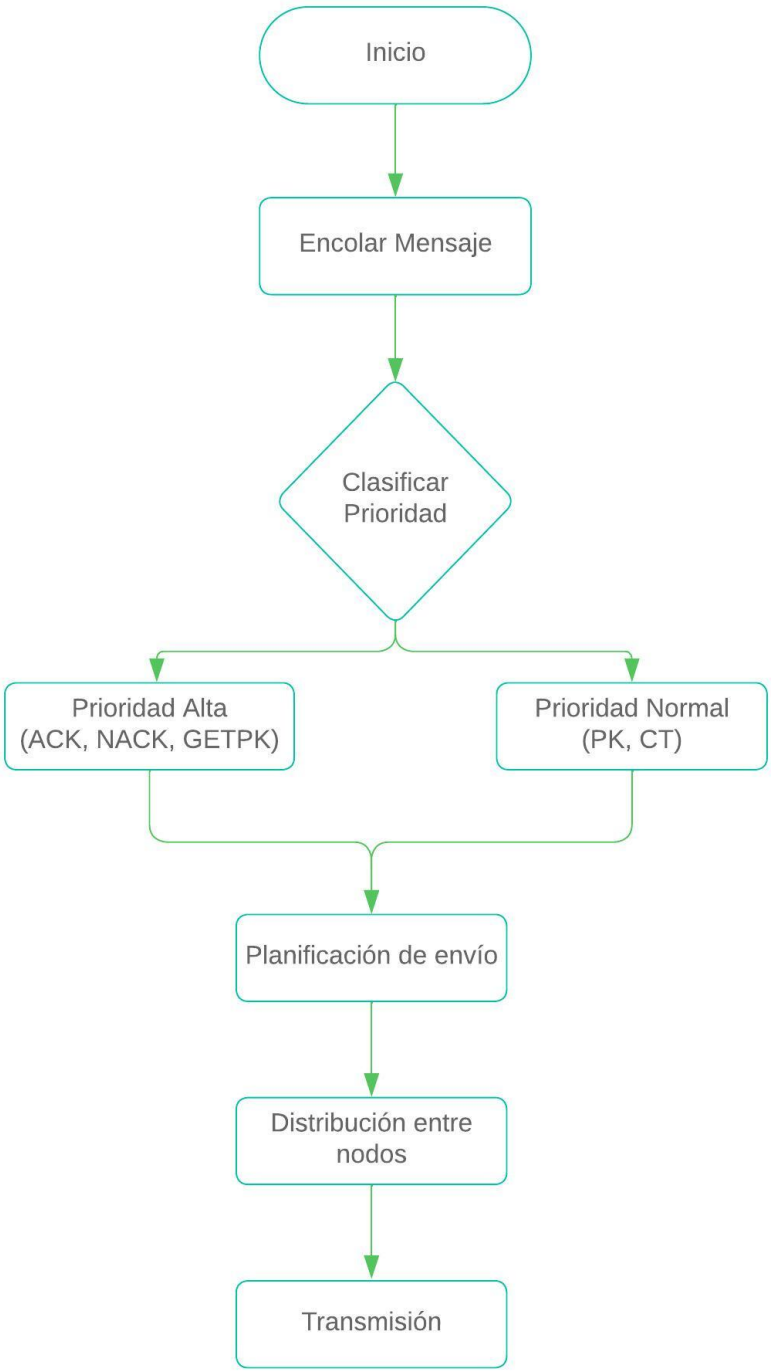


Figura 3.8: Arquitectura de transferencia de mensajes.

3.4 Sesiones

Para poder establecer una topología de red con varios nodos activos, es necesario implementar un mecanismo de control de sesiones, con el objetivo de establecer de forma independiente la comunicación entre cada cliente y el sevidor. Cada sesión esta asociada a un cliente mediante su identificador de nodo (node_id), almacenando los estados de cada sesión, las claves derivadas, contadores e información necesaria para establecer la comunicación.

Tabla 3.5: Sesiones

Categoría	Información almacenada	Función
Identificación	node_id, peer_addr	Identificar al cliente y su dirección IPv6.
Control	role, state, established	Determinar el papel del dispositivo y el estado de la sesión.
Criptografía	ss, aes_key	Mantener el material criptográfico.
Contadores	tx_counter, rx_counter	Gestionar el cifrado y descifrado de datos.
Recuperación	retry_count, retry_deadline	Gestionar los reintentos y sus temporizadores.
Actividad	last_activity	Registrar el instante de la última actividad
Métricas	metrics	Almacenar información experimental.

En el caso del cliente, dispone de una sesión local que almacena toda la información necesaria para establecer la comunicación con el servidor. En el caso del servidor, mantiene una lista de sesiones registradas, con un máximo de 10 sesiones. Cuando el sevidor recibe la GETPK busca la existencia de una sesión asociada a ese identificador. Durante el intercambio, la información almacenada va cambiando, después de recibir el GETPK, el servidor transmite el PK y espera el CT, cuando el cliente encapsula almacena el secreto compartido, y seguidamente la clave derivada. En el servidor sucede un proceso similar.

Después de establecer la sesión, cada vez que el cliente desee enviar un dato, recupera la clave AES almacenada en su sesión, y se inicia el proceso de cifrado, luego encola el mensaje cifrado y lo envia, incrementando el contador de transmisión (tx_counter) cuando el mensaje se ha encolado correctamente. El sevidor recibe el mensaje TEMP, identifica la sesión asociada al nodo emisor, recupera el SS y la clave AES de esa sesión, completa el proceso de descifrado, e incrementa el contador de recepción de mensaje (rx_counter). La sesión pasa a un estado *FAILED* cuando el establecimiento no puede completarse, por ejemplo, al agotar los reintentos permitidos. Se puede reiniciar la sesión, conservando los datos de identificación del nodo.

Este mecanismo permite gestionar simultáneamente múltiples clientes, manteniendo el estado del protocolo de forma indepentiente, y el material criptográfico de cada uno.

3.5 Cifrado de datos

Una vez establecida la sesión entre cliente y servidor, ambos nodos comparte el secreto compartido *Secret Shared SS*, que cunstituye la base para el cifrado de los datos obtenidos

por los sensores. Del Secreto compartido se deriva una clave simétrica de 256 bits.

La implementación del cifrado se realiza mediante el algoritmo AES-256, en modo *CTR* (Counter Mode), que permite cifrar y descifrar bloques de datos de manera eficiente. Los datos ambientales se representan en una estructura que contiene temperatura, humedad y un contador. Antes de enviarlos, se cifran mediante la clave AES derivada del secreto compartido, generando para cada mensaje un vector de inicialización (IV) y un contador de transmisión.

Listing 3.1: Operación de cifrado de datos

```
crypto_worker_aes_ctr_xcrypt(
    s->aes_key,
    iv,
    &packet[PQC_HEADER_LEN],
    sizeof(payload)
);
```

La clave AES se deriva del secreto compartido, como se menciono anteriormente, mediante HQC utilizando la función SHAKE256, que permite generar una clave de 256 bits a partir del secreto compartido.

Listing 3.2: Derivación de la clave AES

```
void crypto_worker_derive_aes_key(const uint8_t *ss, uint8_t *
   aes_key)
{
    if(ss == NULL || aes_key == NULL) {
        return;
    }

    shake256(aes_key, CW_AES_KEY_LEN, ss, CW_HQC_SHARED_LEN);
}
```

SHAKE256 es una función criptográfica de salida extensible (XOF) de la familia SHA-3, que a diferencia de una función de hash con salida fija, permite generar salidas de longitud variable. De esta función se deriva la clave AES de 256 bits, que se utiliza para cifrar los datos obtenidos.

El emisor a partir del SS deriva la clave AES usando la función SHAKE256, cada sesión mantiene un contador que se incrementa al encolar los mensajes correctamente cifrados, a partir de este contador y el SS se genera un vector de inicialización (IV) único para cada mensaje, aplicando SHAKE256 sobre la concatenación del SS y el contador. El IV proporciona el valor inicial del contador interno de AES-CTR, AES cifra los sucesivos bloques del contador utilizando la clave derivada y los bloques generados por AES constituyen el flujo de clave (Keystream), este flujo se combina mediante la operación XOR con el texto plano P_i.

$$C_i = P_i \oplus AES_K(CTR_i) \tag{3.1}$$

En el receptor se realiza un procedimiento equivalente para recuperar el mensaje original, a partir del SS se obtiene la clave AES derivada usando la función SHAKE256, obteniendo la misma clave que el emisor. Se genera el IV a partir del SS y un contador esperado de recepción. Para descifrar el mensaje, se aplica la operación XOR entre el texto cifrado C_i y el flujo de clave generado por AES, obteniendo el mensaje original P_i.

$$P_i = C_i \oplus AES_K(CTR_i) \tag{3.2}$$

Para garantizar la coincidencia de los contadores, se comprueba que el contador esperado coincide con el contador recibido, si es correcto se procesa el mensaje, en caso contrario se prueban hasta 16 valores consecutivos del contador, para recuperar la sincronización.

3.6 Sistema de adquisición de métricas

3.7 Corrección del subsistema de comunicaciones

Una parte muy importante del proyecto fue empleada en la corrección del código de la radio de la EFR32. Durante las campañas de experimentos se detectaron reinicios inesperados en los nodos, especialmente aquellos que desempeñaban funciones de enrutamiento, comprometiendo la estabilidad de las topologías dificultando la ejecución de las campañas.

Los reinicios por watchdogs sucedían en momentos especificos de las pruebas, cuando los nodos recibian cargas importantes de paquetes, para entender el problema, se propuso un experimento simple, dos nodos en línea con el servido. Se compilo un codigo de ejemplo sin carga criptográfica en el nodo intermedio, tanto en el nodo externo como en el servidor se compilo el código de HQC.

Al iniciar el intercambio de claves entre el nodo exterior y el servidor, rapidamente se reinicio el nodo intermedio, descartando así la autoría de los reinicios a la carga crriptografica de HQC. Para poder diagnosticar correctamente el incidente, fue necesario analizar el código del driver de la radio del microcontrolador. Dentro del código, salta a la vista la función *move_to_rx_buffer()*, en ella se toman los paquetes recibidos por el hardware de radio RAIL, lo valida y luego los copia a la cola de recepción del driver, finalmente libera el paquete interno de RAIL.

Existe una condición de carrera en el momento que la función *move_to_rx_buffer()*, ejecutandose desde el callback de RAIL, intenta reservar ó publicar un buffer, mientras que el proceso principal de contiki-ng consulta, extrae o libera esos mismos buffers, el driver intenta hacer:

Listing 3.3: Sección crítica original para comprobar la disponibilidad de un buffer RX y actualizar el índice de escritura.

```
if(rx_buf[nw = next_write].len == 0) {
  next_write = (next_write + 1) % RX_BUF_COUNT;
  return &rx_buf[nw];
}
```

Esta operación contiene varios pasos:

1. Leer next_write
2. Calcular rx_buffer[next_write]
3. Lees len
4. Comprorbar si len = 0
5. Calcular el nuevo índice
6. Escrribir next_write
7. Devolver el puntero

Contiki puede comenzar a liberar o consultar un buffer, luego una interrupción de radio suspende a contiki en mitad de esta operación, modifica la cola en ese estado intermedio, termina la interrupción y contiki continúa desde donde fue interrumpido, usando la información que puede haber quedado desactualizada. La consecuencia inmediata no tiene por qué ser corrupción de memoria, también puede pasar por pérdidas de paquetes, lecturas incorrectas de los buffers, desincronización de los índices, y estas inconsistencias provocan una copia con longitud incorrecta, o un bloqueo produciendo un Watchdog reset.

3.7.1 Modificacion del driver

La solución propuesta pasa por proteger las operaciones, duranet la sección mencionada, el callback y el proceso principal no pueden intercalarse a mitad de una operación. Para ello se protegen con las macros *CORE_DECLARE_IRQ_STATE*, *CORE_ENTER_CRITICAL*() y *CORE_EXIT_CRITICAL*(), aplicandose a las consultas de paquetes, extracciones, reservas y liberación de buffers.

Listing 3.4: Protección atómica de la reserva de un buffer RX mediante una sección crítica.

```
CORE_DECLARE_IRQ_STATE;
CORE_ENTER_CRITICAL();

if(rx_buf[nw = next_write].len == 0) {
  next_write = (next_write + 1) % RX_BUF_COUNT;
  result = &rx_buf[nw];
}

CORE_EXIT_CRITICAL();
```

CORE_ENTER_CRITICAL() deshabilita temporalmente las interrupciones y guarda su estado anterior, de esta manera toda la secuencia se ejecuta como una sola operación. Después *CORE_EXIT_CRITICAL*() restaura el estado anterior de las interrupciones, esto impide que el callback de RAIL interrumpa el proceso, mientras existan modificaciones en cola. Así, ningun contexto puede ver *next_write* antes de que termine la reserva lógica del buffer.

3.8 Diseño experimental

3.8.1 Escenarios y topologías

3.8.2 Condiciones experimentales

3.8.3 Métricas evaluadas

3.8.4 Procedimiento experimental

Capítulo 4

Resultados

En este capitulo se presentaran los resultados obtenidos durante las campañas de expermientos, cuyo objetivo es el de evaluar el impacto de la integración de HQC sobre una red IoT formada por sistemas embebidos.

El analisis se basa en estudiar el comportamiento del sistema bajo diferentes condiciones experimentales como se reviso en el capitulo anterior. Los resultados estan organizados por topologías, comenzando con la configuración P2P, y continuando con Árbol, Linea y Estrella. Finalmente se realiza una comparación global entre las diferentes topologías, con el proposito de identificar las principales diferencias observadas.

Para obtener los datos, se realizaron 3 pruebas por condición, y se realizo una media ponderada para cada condición de cada topologia.

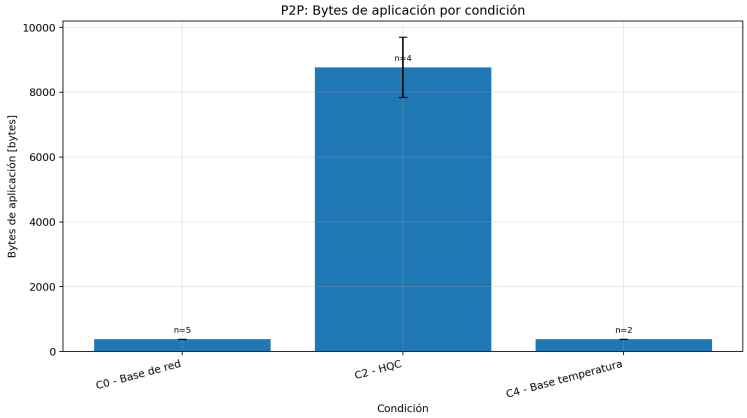
4.1 P2P

Para comenzar con la campaña de pruebas, se ejecutaron las condiciones de prueba 0, 2 y 4 sobre la topología P2P, en la que se busca conocer el impacto de transportar HQC entre dos nodos, o nodo y servidor.

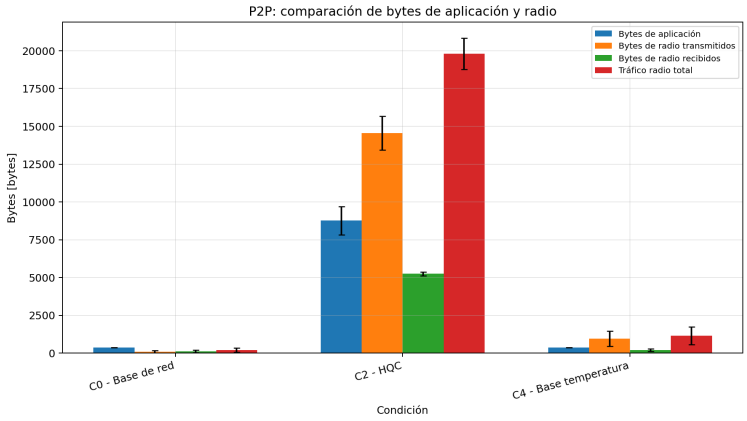
En la Figura 4.1a se puede observar como, en la condición 2 se genera un tráfico masivo, alcanzando los 8KB en promedio. Esto significa que el transporte de las claves publica y privada son en mayor medida el gran cuello de botella de el protocolo. A diferencia de la condición 4, en la que el cifrado de los datos a penas afecta al tráfico entre los nodos.

La Figura 4.1b muestra una capa nueva, durante el establecimiento de la red segura con HQC, se registraron 8KB aproximadamente, mientras que los contadores de radio registraron 14KB transmitidos. La diferencia puede atribuirse tanto al efecto de la sobercarga en las comunicaciones, y a las posibles retransmisiones, aunque no ha sido posible cuantificar individualmente estas contribuciones.

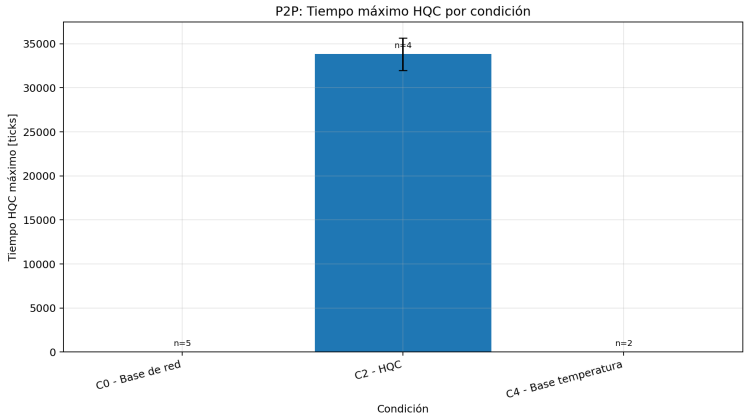
Por otro lado, tenemos un tiempo medio de 33000 Ticks aproximadamente en establecer la red segura, asi como el procesamiento matematico de la cpu, según nos marca la Figura 4.1c. Este comportamiento es el esperado, debido al tamaño de las claves transportadas.



(a) Bytes de la app



(b) Comparación tx rx

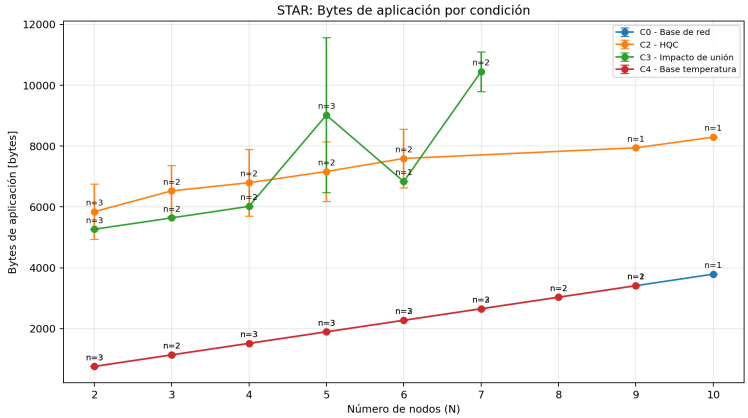


(c) Tiempo HQC

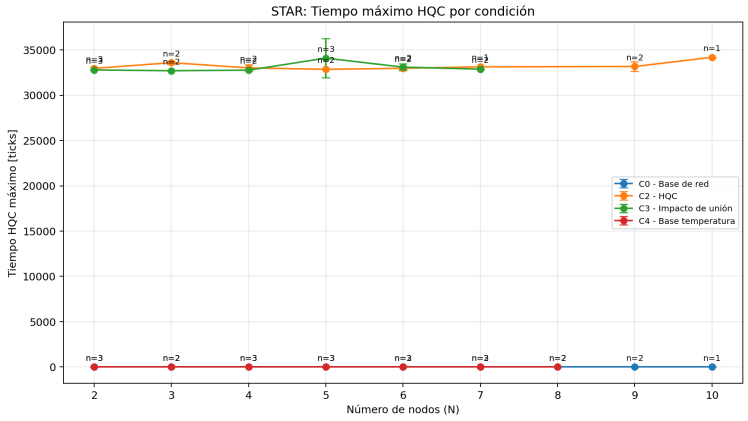
Figura 4.1: Análisis global del tráfico generado en escenarios P2P

4.2 Estrella

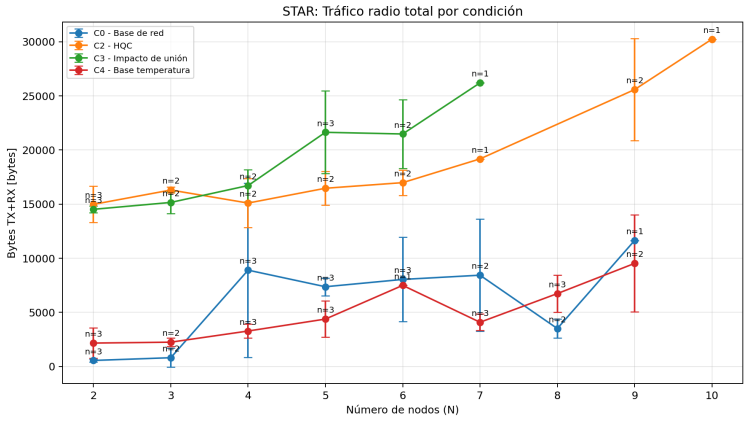
El siguiente paso es realizar la campaña de pruebas, se ejecutan las las condiciones de pruebas 0, 2, 3 y 4 sobre la topología estrella, las primeras tres figuras generadas con las siguientes.



(a) Bytes de la app



(b) Tiempos totales por condición



(c) Tráfico total por condición

Figura 4.2: Análisis global del tráfico generado en escenarios estrella

Como se puede observar en la Figura 4.2a, las condiciones 0 y 4 tienen un comportamiento lineal, creciendo a medida que crece la red, es normal esperar este comportamiento, debido a que crece la dificultad de mantener la red con el crecimiento de la misma. Con respecto a la condicion 2, se tiene un comienzo relativamente alto, al tener 2 nodos sin contar con el servidor, y esperable debido a que no existen saltos intermedios, sin embargo, al crecer la red, crece con una pendiente menor que la condición 4.

Existen anomalias en la condición 3, teniendo en cuenta que en esta condición se pretende estudiar el impacto que tiene la unión en una red establecida junto con trafico de datos cifrados, con una cantidad pequeña de nodos tiene un comportamiento similar a la condición 2, lo que representa un comportamiento normal, debido a que los canales de comunicación estan despejados. A partir del 5 nodo, existe una saturación del canal, por lo que el servidor tiene que gestionar las comunicacione existentes mientras procesa la unión del nuevo nodo y sus datos.

Los tiempos de establecimiento de la comunicación segura son estables durante todas las condiciones y numero de nodos, como muesta la Figura 4.2b, como es de esperar, al no tener saltos intermedios, no existe penalización de retransmisión entre nodos.

Por otro lado, es importante mencionar el tráfico de radio total de la Figura 4.2c, donde se observa que el tráfico en la condición 3 para 5 nodos se dispara a más de 20KB aproximadamente, demostrando que el tráfico de radio en esta condición támbien se debe a retransmisiones y paquetes de control. Al añadir un nodo extra, junto con los datos cifrados, el medio físico se congestiona, provocando que el protocolo deba reenviar paquetes.

El crecimiento eponencial de la condición C2, revelando el coste en escalabilidad de HQC en una topología estrella. Aunque el protocolo intente mantener un flujo controlado, el servidor sufre un desgaste en el uso de la radio.

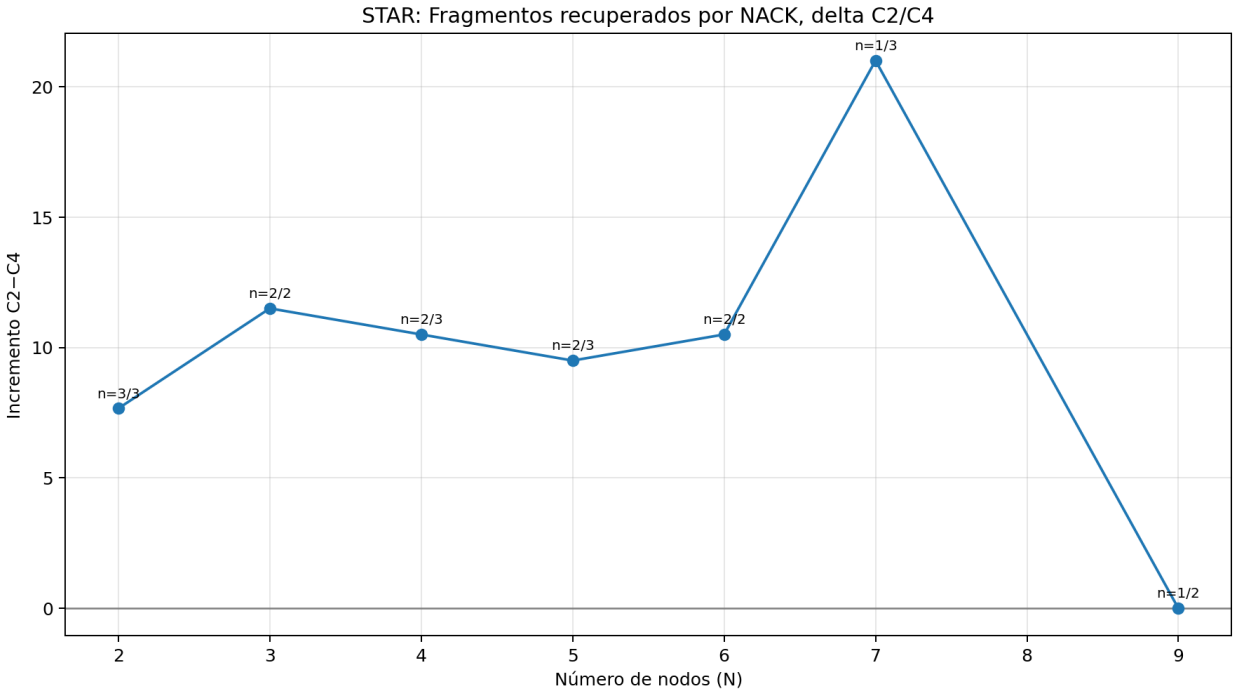


Figura 4.3: Fragmentos recuperados NACK

Para terminar de completar la imagen de la topología estrella, la Figura 4.3, demuestra el $\Delta C_2/C_4$ en fragmentos recuperados por el NACK. Los resultados revelan una correlación con las metricas de trafico: al alcanzar los 7 nodos, el sistema sufre un pico crítico de más de 21 fragmentos perdidos y recuperados por NACK. Esto demuestra que el tamaño de las claves del algoritmo HQC satura el medio físico de la topología, provocando colisiones masivas y forzando al protocolo a realizar retransmisiones. El escenario de 9 nodos y su desplome denota una situación de saturación donde la congestion impide la correcta señalización de paquetes perdidos.

4.3 Línea

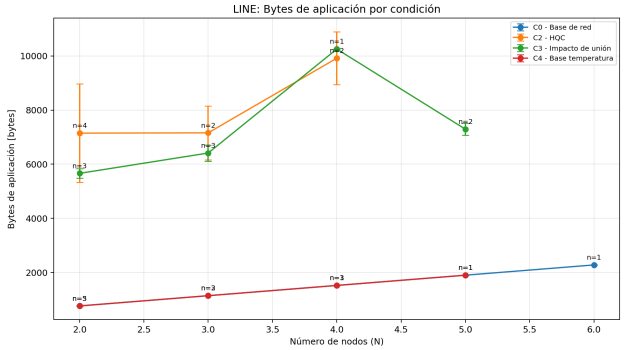
En esta sección se presentan los resultados obtenidos durante las pruebas realizadas en la topología línea, que, a diferencia de las configuraciones P2P y estrella, donde los nodos realizan una conexión directa con el servidor, esta topología introduce la participación de nodos intermedios, que se encargan de retransmitir los paquetes a través de múltiples saltos hasta el servidor.

El análisis de la Figura 4.4a permite analizar el comportamiento de cada condición en función del crecimiento de la red. Al observar la condición 0 y 4, salta a la vista el comportamiento lineal de las condiciones, al igual que en estrella, demostrando asi, que una vez asegurado el cifrado de las comunicaciones, el transporte es altamente eficiente.

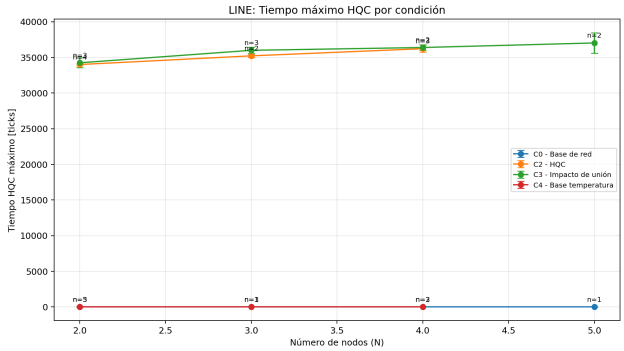
Por la naturaleza de la topología, se entiende el colapso de las comunicaciones, la linea C2 arranca en un nivel alto alcanzando unos 10KB, y a partir del quinto nodo se pierden

por completo los datos, debido a una alta saturación del medio. Teniendo la linea C3 un comportamiento similar.

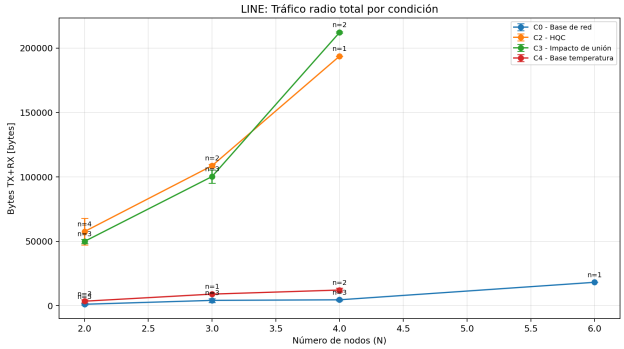
Los tiempos de establecimiento de la red se mantienen estables entre topologías, con una latencia esperable debido a los saltos que deben acometer los paquetes, como lo remarcan las lineas C2 y C3 de la Figura 4.4b. Siguendo el análisis inicial una vez establecida la comunicación segura, es altamente eficiente el intercambio de paquetes cifrados, teniendo los mismos tiempos de esttablecimiento la linea C4 que la linea C0.



(a) Bytes de la app



(b) Tiempos totales por condición



(c) Tráfico total por condición

Figura 4.4: Análisis global del tráfico generado en escenarios linea

La Figura 4.4c complementa perfectamente el análisis realizado anteriormente, a medida que

se añaden capas a la red, el incremento del tráfico es cuasi-exponencial, a partir del cuarto nodo existe una saturación del medio tal que no permite seguir recuperando datos de los nodos involucrados. Esto demuestra que el medio físico sufre una amplificación de más de 20 veces respecto a la aplicación.

En configuraciones de 2 y 3 nodos, el protocolo tolera la carga criptográfica, manteniendo el erscate de paquetes en una meseta de 20 a 25 fragmentos, sin embago, a partir del cuarto nodo, el índice de pérdidas se dispara drásticamente, alcanzando un pico de más de 40 fragmentos recuperados. Este disparo del mecanismo NACK, mostrado en la Figura 4.5, confirma que el tamaño de las claves de HQC satura los búfers de los nodos intermedios, multiplicando las colisiones en el medio físico. Por lo que la topología linea representa el peor escenario en una red IoT, debido a una alta saturación tanto de la capa física como de los nodos involucrados en la conexión.

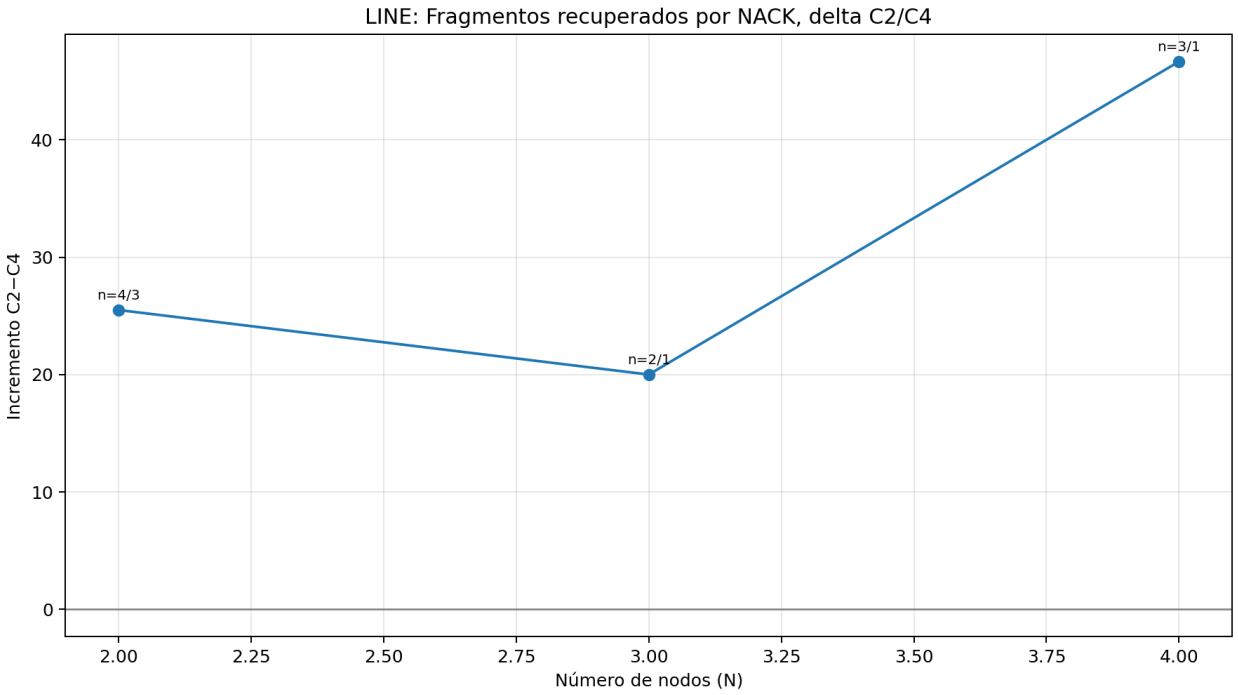


Figura 4.5: Fragmentos recuperados NACK

4.4 Árbol

Está sección presenta los resultados obtenidos en la campaña de pruebas realizada en la topología Árbol, similamente a la topología linea, en está la comunicación entre los nodos más alejados y el servidor requiere la participación de nodos intermedios, sin embargo, no se espera el sobrecoste de la topología linea.

Similarmente a la topología linea, en la Figura 4.6a, los bytes de aplicación en C0 y C4 crecen linealmente con el crecimiento de la red, y al igual que la topología anterior, se tiene una alta eficiencia una vez establecida la comunicación segura.

Existe una mayor resiliencia en el intercambio de claves en comparación con la topología linea, hay una escalabilidad de hasta 9 nodos en Árbol. La jerarquia del árbol divide el táfico, las ramas distribuyen la carga de transmisión de las claves de HQC, retrasando asi el colapso de la red.

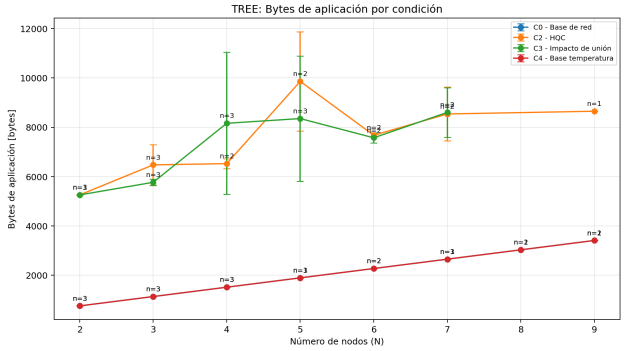
El pico en el quinto nodo donde C2 roza los 10KB, indica que al nivel de ramificación la red sufre una congestión severa, requiriendo multiples reintentos.

Con respecto a los tiempos de establecimiento, Figura 4.6b, siguen la misma trayectoria que topologias anteriores, las lineas C0 y C4 son identicas, demostrando la alta eficiencia en el envio de los paquetes cifrados, y al transmitir las claves existe una latencia evidente.

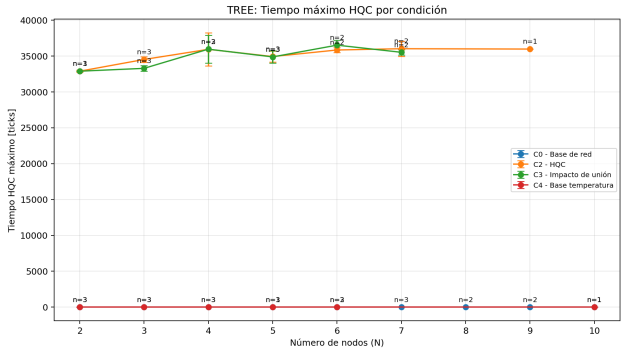
Al observar la Figura 4.6c destaca el disparo en el tráfico generado en la topología, superando facilmente los 110KB generando asi un overhead del 1200 % aproximadamente en el punto más alto. A partir del sexto nodo, los nodos padres experimentan un cuello de botella extremo, al recibir ráfagas de los fragmentos de las claves los bufers se desbordan, provocando perdidas masivas de paquetes.

A partir del septimo nodo, se puede intuir que el sistema alcanza un estado de saturación física, en el que no importa si se intercambian claves o intenta unirse un nuevo nodo, el espectro de radio está congestionado de tal manera que opera al 100 % de su capacidad en reintentos.

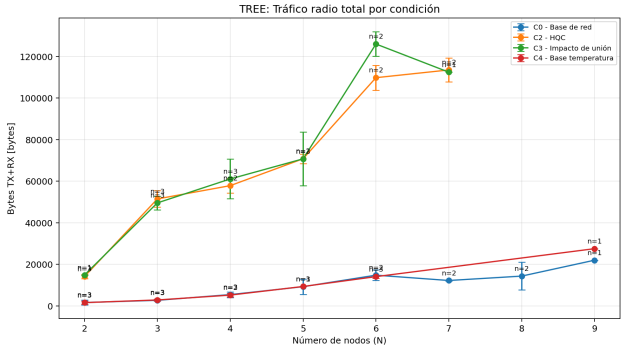
Por otro lado, las lineas C0 y C4 muestran otra cara, manteniendose estables a lo largo del crecimiento de la red, indicando que el problema de la criptografia post-cuánica en estructuras jerárquicas es un problema de establecimiendo de las claves.



(a) Bytes de la app



(b) Tiempos totales por condición



(c) Tráfico total por condición

Figura 4.6: Análisis global del tráfico generado en escenarios linea

La Figura 4.7 evidencia el cuello de botella cerca del quinto nodo, alcanzando un pico de más de 40 fragmentos recuperados por otro lado, al llegar al sexto y septimo nodo cae drasticamente la gráfica, esto puede deberse a la congestión extrema del medio físico de tal manera que los nodos dejan de recibir correctamente los paquetes y evita que se procese la telemetría. El protocolo puede dejar de emitir los NACK, o los nodos dejan de escuchar este protocolo.

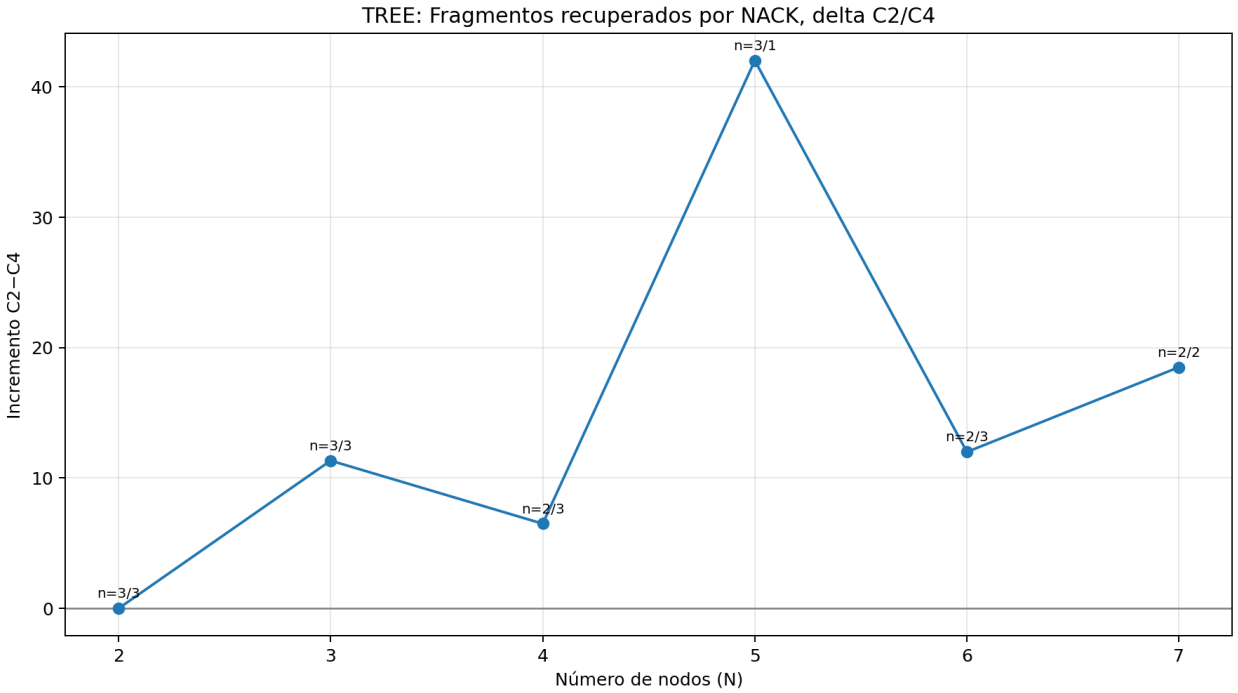


Figura 4.7: Fragmentos recuperados NACK

Capítulo 5

Conclusiones

Bibliografía

[1] N. Kumar, "How many IoT devices are there (2025-2030 data)."

[2] I. T. L. Computer Security Division, "Post-quantum cryptography | CSRC | CSRC."

[3] T. M. Fernandez-Carames, "From pre-quantum to post-quantum IoT security: A survey on quantum-resistant cryptosystems for the internet of things," vol. 7, no. 7, pp. 6457–6480.

[4] "HQC."

[5] O. Kuznetsov, S. Kandiy, E. Frontoni, and O. Smirnov, "Trade-offs in post-quantum cryptography: A comparative assessment of BIKE, HQC, and classic McEliece,"

[6] S. Vaudenay, *A classical introduction to cryptography: applications for communications security*. Springer.

[7] M. Nieles, K. Dempsey, and V. Y. Pillitteri, "An introduction to information security."

[8] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer Berlin Heidelberg.

[9] European Union Agency for Cybersecurity., *Post-quantum cryptography: current state and quantum mitigation.* Publications Office.

[10] M. Schöffel, J. Feldmann, and N. Wehn, "Code-based cryptography in IoT: A HW/SW co-design of HQC," in *2022 IEEE 8th World Forum on Internet of Things (WF-IoT)*, pp. 1–7, IEEE.

[11] "Hamming quasi-cyclic (HQC) fourth round version updated version 01/10/2022,"

[12] E. White, *Making embedded systems: design patterns for great software*. O'Reilly.

[13] T. S. Rappaport, *Wireless communications: principles and practice*. Prentice Hall communications engineering and emerging technologies series, Prentice Hall, 2. ed., 18. printing ed.

[14] T. Liu, G. Ramachandran, and R. Jurdak, "Post-quantum cryptography for internet of things: A survey on performance and optimization."

[15] M. Krausz, G. Land, J. Richter-Brockmann, and T. Güneysu, "A holistic approach towards side-channel secure fixed-weight polynomial sampling," in *Public-Key Cryptography – PKC 2023* (A. Boldyreva and V. Kolesnikov, eds.), vol. 13941, pp. 94–124, Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science.

[16] S. Deshpande, C. Xu, M. Nawan, K. Nawaz, and J. Szefer, "Fast and efficient hardware implementation of HQC," in *Selected Areas in Cryptography – SAC 2023* (C. Carlet, K. Mandal, and V. Rijmen, eds.), vol. 14201, pp. 297–321, Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science.

[17] "HQC: Hamming quasi-cyclic an IND-CCA2 code-based public key encryption scheme,"

[18] C. Aguilar-Melchor, J.-C. Deneuville, A. Dion, J. Howe, R. Malmain, V. Migliore, M. Nawan, and K. Nawaz, "Towards automating cryptographic hardware implementations: A case study of HQC," in *Code-Based Cryptography* (J.-C. Deneuville, ed.), vol. 13839, pp. 62–76, Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science.

[19] J.-A. Septien-Hernandez, M. Arellano-Vazquez, M. A. Contreras-Cruz, and J.-P. Ramirez-Paredes, "A comparative study of post-quantum cryptosystems for internet-of-things applications," vol. 22, no. 2, p. 489.

[20] S. P C, K. Jain, and P. Krishnan, "Analysis of post-quantum cryptography for internet of things," in *2022 6th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pp. 387–394, IEEE.

[21] S. Maurya, S. Kasture, and A. Shukla, "Quantum cryptography for secure communications in industrial mechatronics and embedded systems," in *2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*, pp. 1–8, IEEE.

[22] N. Aragon, P. L. Barreto, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu, C. A. Melchor, R. Misoczki, E. Persichetti, J. Richter-Brockmann, N. Sendrier, J.-P. Tillich, V. Vasseur, and G. Zémor, "5th NIST PQC standardization workshop april 10th, 2024,"

[23] L. Demange and M. Rossi, "A provably masked implementation of BIKE key encapsulation mechanism," pp. cc1–1–57.

[24] N. Aragon, "BIKE: Bit flipping key encapsulation,"

[25] M.-S. Chen, T. Chou, and M. Krausz, "Optimizing BIKE for the intel haswell and ARM cortex-m4," pp. 97–124.

[26] J. Richter-Brockmann, M.-S. Chen, S. Ghosh, and T. Güneysu, "Racing BIKE: Improved polynomial multiplication and inversion in hardware," pp. 557–588.

[27] R. Urian, "Classic McEliece key generation on RAM constrained devices,"

[28] M.-S. Chen and T. Chou, "Classic McEliece on the ARM cortex-m4," pp. 125–148.

[29] T. Hemmert, A. May, J. Mittmann, and C. R. T. Schneider, "How to backdoor (classic) McEliece and how to guard against backdoors," in *Post-Quantum Cryptography* (J. H. Cheon and T. Johansson, eds.), vol. 13512, pp. 24–44, Springer International Publishing. Series Title: Lecture Notes in Computer Science.

[30] J. Samandari and D. C. Gritti, "McEliece post quantum cryptography for IoT,"

[31] P. Pérez-Pacheco and P. Caballero-Gil, "McEliece cryptosystem: Reducing the key size with QC-LDPC codes," in *2023 19th International Conference on the Design of Reliable Communication Networks (DRCN)*, pp. 1–6, IEEE.

[32] P.-J. Chen, T. Chou, S. Deshpande, N. Lahr, R. Niederhagen, J. Szefer, and W. Wang, "Complete and improved FPGA implementation of classic McEliece," pp. 71–113.

[33] F. Ivanov, E. Krouk, and A. Kreshchuk, "On the lightweight McEliece cryptosystem for low-power devices," in *2019 XVI International Symposium "Problems of Redundancy in Information and Control Systems"(REDUNDANCY)*, pp. 133–138, IEEE.

[34] M. Sim, S. Eum, H. Kwon, H. Kim, and H. Seo, "Optimized implementation of encapsulation and decapsulation of classic McEliece on ARMv8,"

[35] A. Kuznetsov, M. Lutsenko, M. Bagmut, and V. Zhora, "Performance evaluation of the classic McEliece key encapsulation algorithm," in *2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, pp. 755–760, IEEE.

[36] E. Bindal and A. K. Singh, "Secure and compact: A new variant of McEliece cryptosystem," vol. 12, pp. 35586–35596.

[37] A. Wakasugi and M. Tada, "Security analysis for BIKE, classic McEliece and HQC against the quantum ISD algorithms,"

[38] Y. Li and L.-P. Wang, "Security analysis of the classic McEliece, HQC and BIKE schemes in low memory," vol. 79, p. 103651.

[39] R. Avanzi, S. Hoerder, D. Page, and M. Tunstall, "Side-channel attacks on the McEliece and niederreiter public-key cryptosystems," vol. 1, no. 4, pp. 271–281.

[40] "Classic McEliece: Intro."

[41] W. Stallings, "Cryptography and network security: Principles and practice,"

[42] F. Hu, "Security and privacy in internet of things (IoTs): Models, algorithms, and implementations,"

[43] H. Aldowah, S. Ul Rehman, and I. Umar, "Security in internet of things: Issues, challenges and solutions," in *Recent Trends in Data Science and Soft Computing* (F. Saeed, N. Gazem, F. Mohammed, and A. Busalim, eds.), vol. 843, pp. 396–405, Springer International Publishing. Series Title: Advances in Intelligent Systems and Computing.

[44] J. F. Kurose and K. W. Ross, *Computer networking: a top-down approach*. Pearson Education, 7. edition ed.

[45] R. Overbeck and N. Sendrier, "Code-based cryptography," in *Post-Quantum Cryptography* (D. J. Bernstein, J. Buchmann, and E. Dahmen, eds.), pp. 95–145, Springer Berlin Heidelberg.

[46] J. L. Hernandez-Ramos, J. A. Martinez, V. Savarino, M. Angelini, V. Napolitano, A. F.

Skarmeta, and G. Baldini, “Security and privacy in internet of things-enabled smart cities: Challenges and future directions,” vol. 19, no. 1, pp. 12–23.

Anexos

Incluye este apartado solo si es necesario. En caso contrario, comenta en el fichero principal (main.tex) las líneas `\appendix` y `\includecuerpo/x-Anexos` para evitar su inclusión.